

LangGraph

LLM을 좀 더 논리적으로 동작하도록...

Hunter Shin [hunter.shin@gmail.com]

LangGraph 학습 내용

- ① LangGraph 는 어떤 프레임워크 일까?
- ② State, Node, Edge, Conditional Edge
- ③ Workflow와 Agent 설계 관점
- ④ State Reducer, Command, Send, 병렬 실행
- ⑤ Memory, Checkpoint, Time Travel, Durable Execution
- ⑥ ReAct, Self-RAG, Corrective RAG, HITL
- ⑦ 운영 관점: 오류 처리, Trace, 테스트, Guardrail

LLM/RAG 사용시 발생하는 이슈



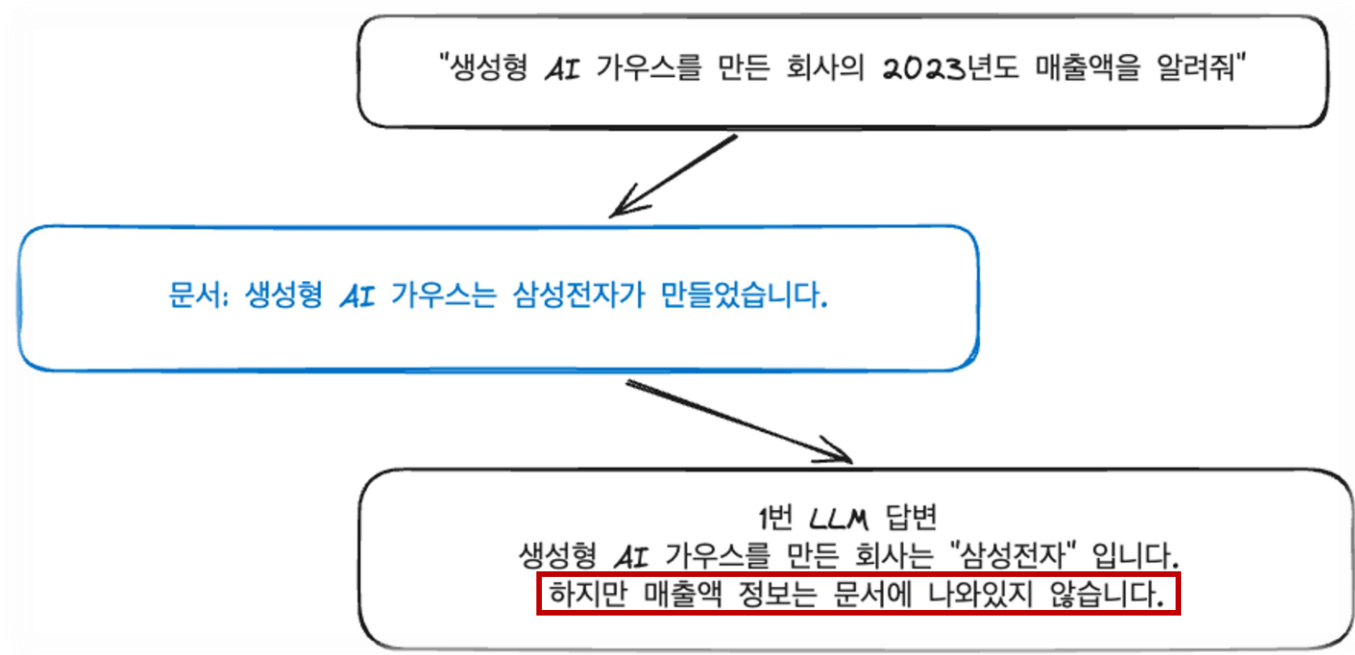
RAG 사용의 한계

RAG(Retrieval-Augmented Generation) 이라는 강력한 기능을 갖게 된 우리는 한 번쯤 다음의 갈등을 마주하게 됩니다.

- LLM 이 생성한 답변이 Hallucination 이 아닐까?
- RAG 를 적용하여 받은 답변이 문서에는 없는 “사전지식”으로 답변한 건 아닐까?
- “문서 검색에서 원하는 내용이 없을 경우”
 - > “인터넷” 혹은 “논문” 에서 부족한 정보를 검색하여 지식을 보강할 수는 없을까?

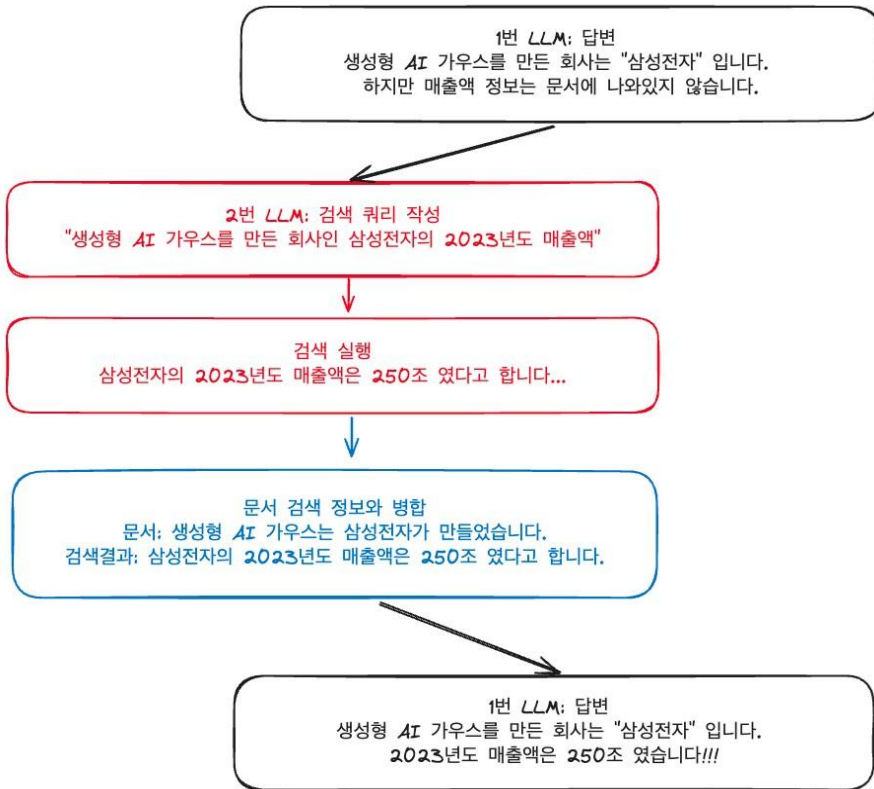
RAG 개발 단계에서의 고민 사례 #1

일반 RAG 를 수행했을 때 흔히 볼 수 있는 상황 (문서 내 질문에 대한 답변이 존재하지 않는 경우)



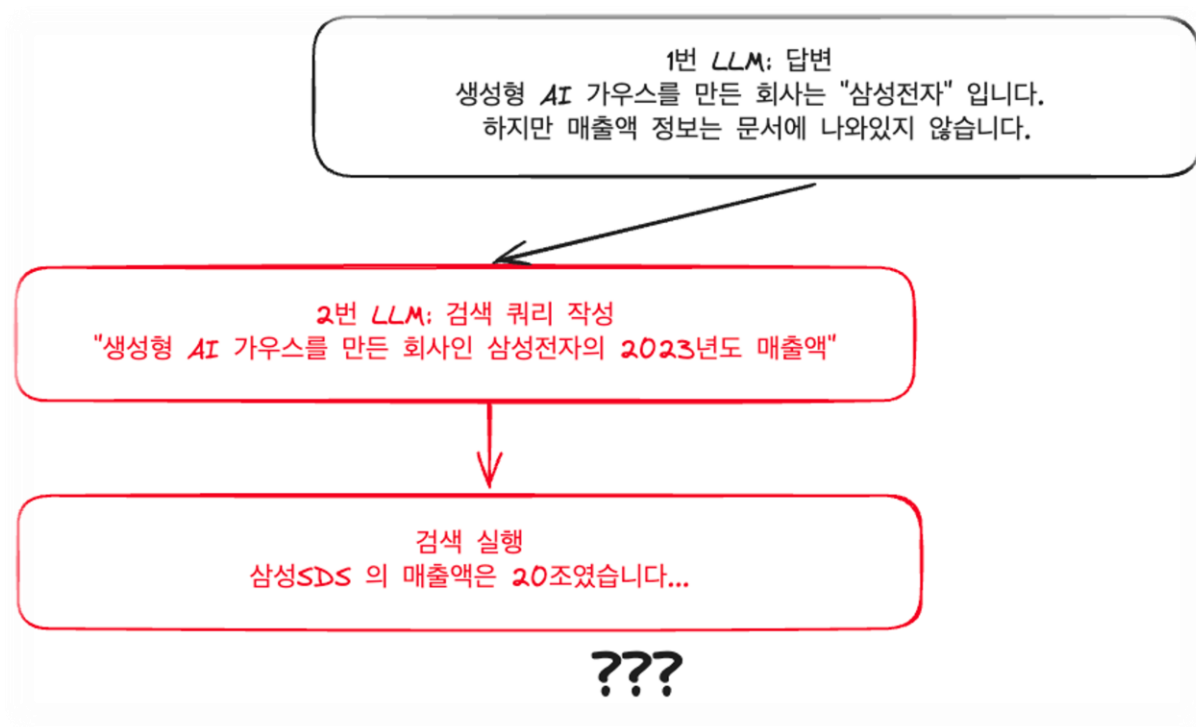
RAG 개발 단계에서의 고민 사례 #2

그러면 부족한 정보를 Web 검색하여 문서에 추가하는 로직을 추가해서 검색을 진행할 수 있음



RAG 개발 단계에서의 고민 사례 #3

만약 검색결과에 잘못된 정보가 포함되거나 혹은 검색 결과에 없다면?



RAG 개발 단계에서의 고민 사례 #4

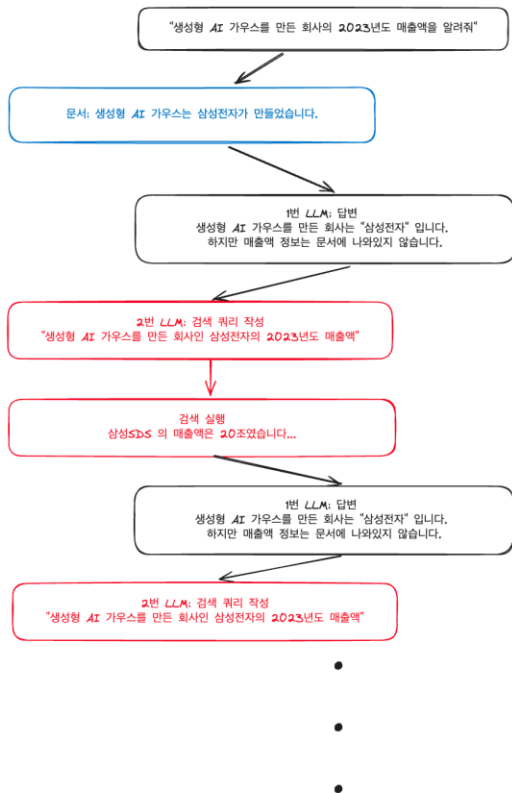
잘못된 검색결과가 결국 **Hallucination** 이 이어진다면?

1번 LLM: 답변
생성형 AI 가우스를 만든 회사는 "삼성SDS" 입니다.
2023년도 매출액은 20조 였습니다.

???

RAG 개발 단계에서의 고민 사례 #5

결국 코드가 점점 길어지고 복잡해지며 LLM 의 일관되지 않은 답변이 마치 나비효과로 이어져 답변 품질저하로 이어짐



- 검색이 제대로 나올 때까지 반복하여 검색 해볼까? (반복문)
 - 무한히 제대로 된 결과가 안 나올 경우 토큰 사용량이 폭증 할텐데?
- Hallucination 을 방지하는 LLM 을 추가해야 하나?

기본적인 RAG의 문제점

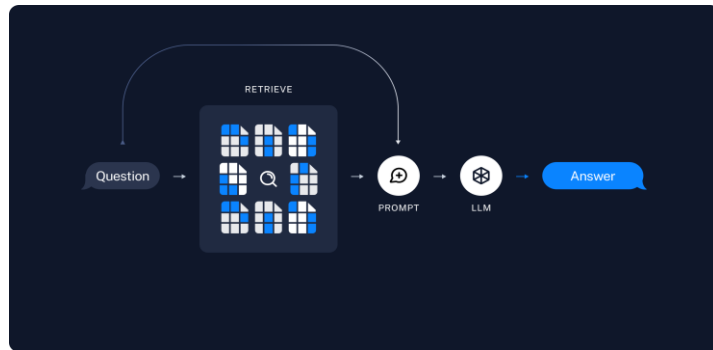
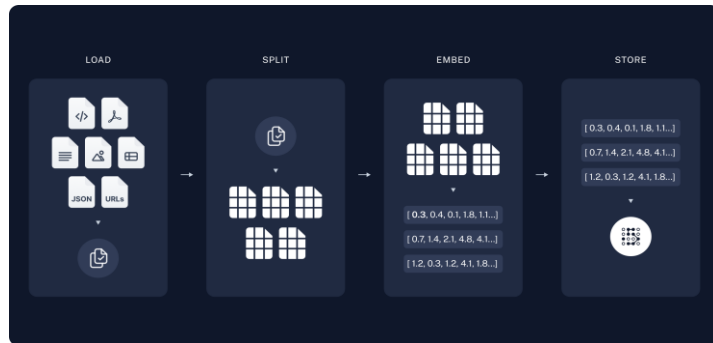
RAG는 모든 것들이 사전에 정의된 것에 의해서 동작

- 사전에 정의된 데이터 소싱(PDF, DB, Table 등) 자원
- 사전에 정의된 Fixed Size Chunk
- 사전에 정의된 Query 입력
- 사전에 정의된 검색 방법
- 신뢰하기 어려운 LLM 혹은 Agent
- 고정된 프롬프트 형식
- LLM의 답변 결과에 대한 문서와의 관련성/신뢰성

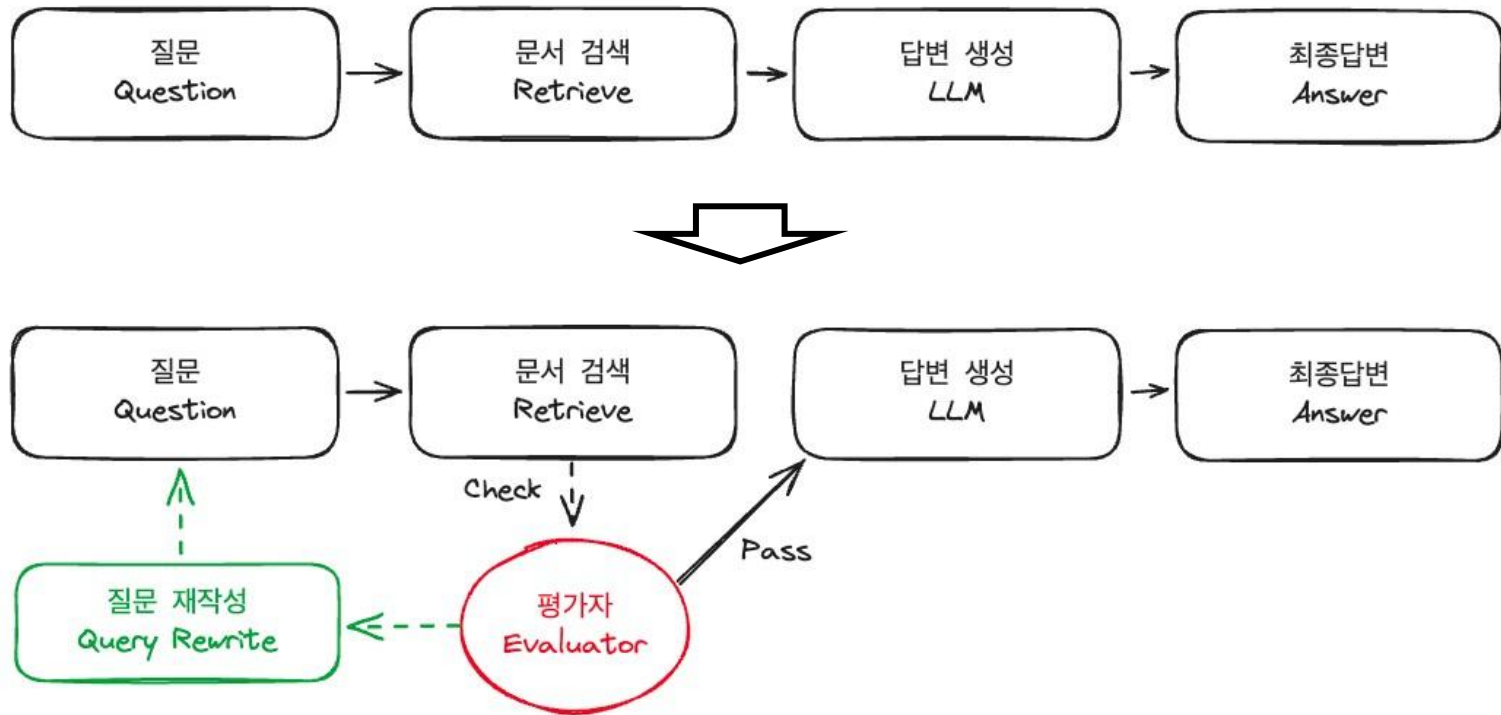
Document Loader(데이터로드) > Answer(답변)

RAG 파이프라인이 단방향 구조이기 때문

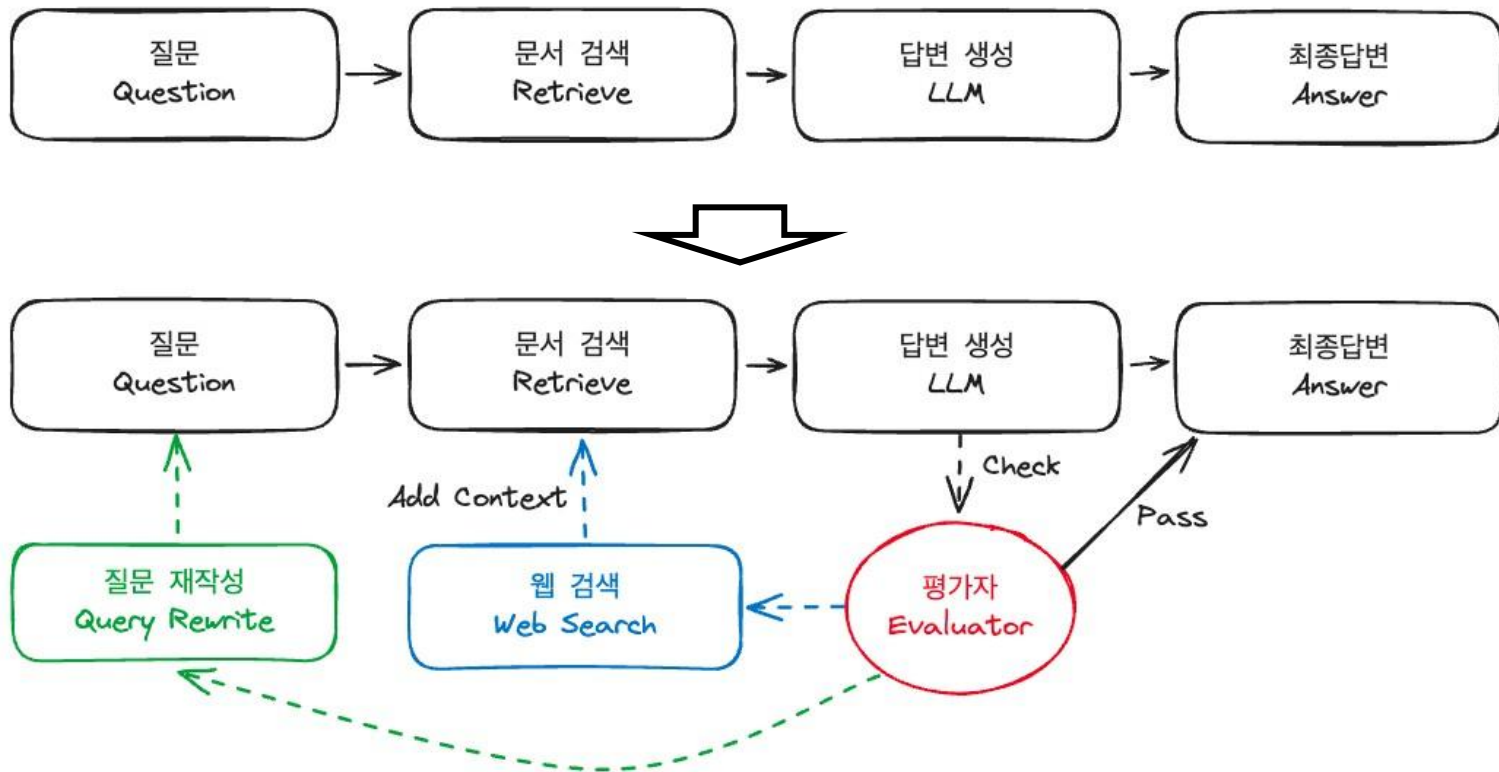
- 모든 단계를 한 번에 다 잘해야 함
- 이전 단계로 되돌아가기 어려움
 - 이전 과정의 결과물을 수정하기 어려움



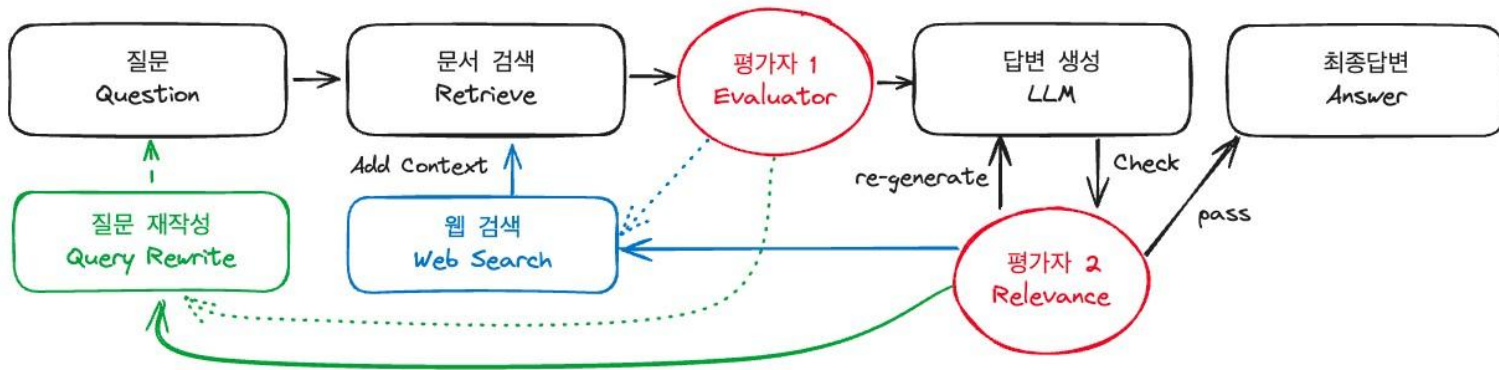
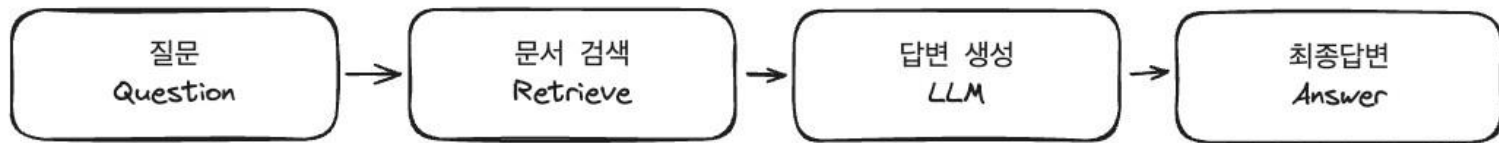
IDEA #1 | 평가자 & Query Transform 추가



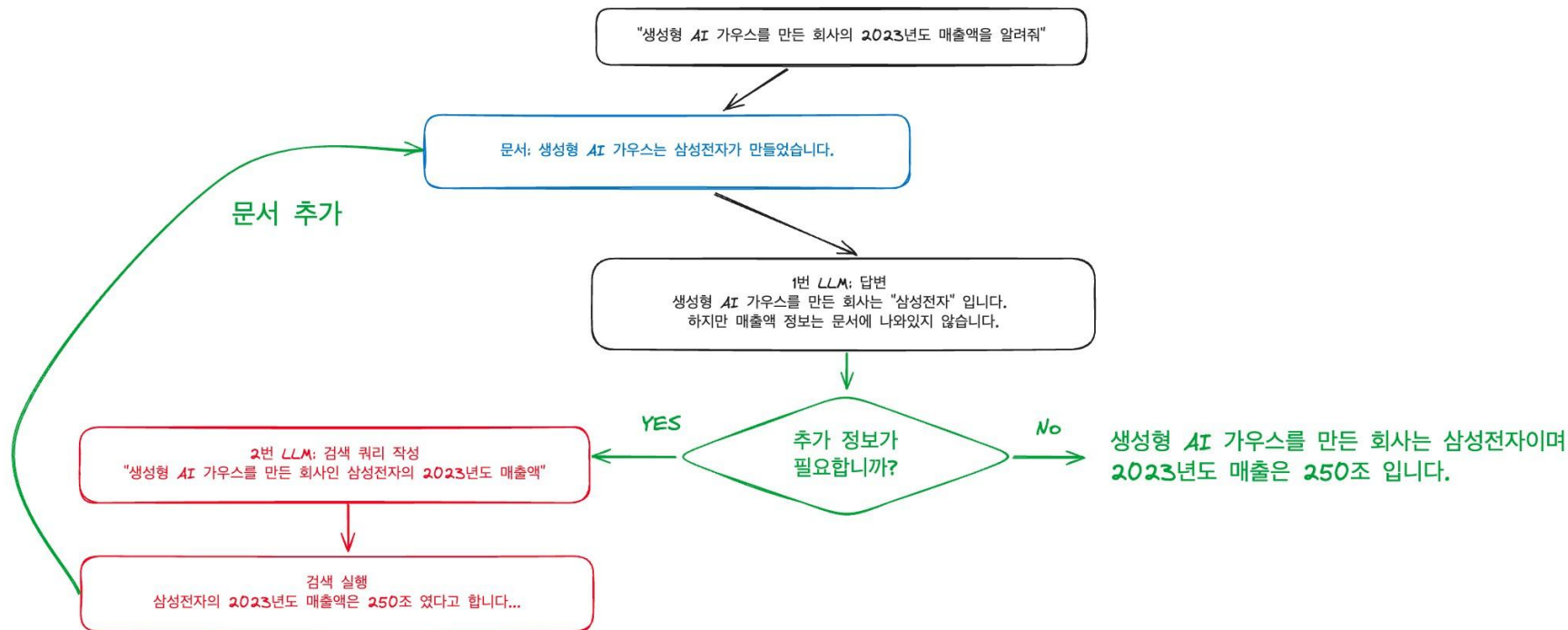
IDEA #2 | 추가 검색기를 통하여 문맥 (context) 보강



IDEA #3 | 문서-답변 간 관련 여부를 판단하는 평가자2 를 추가 검증



해당 아이디어의 구현 예시

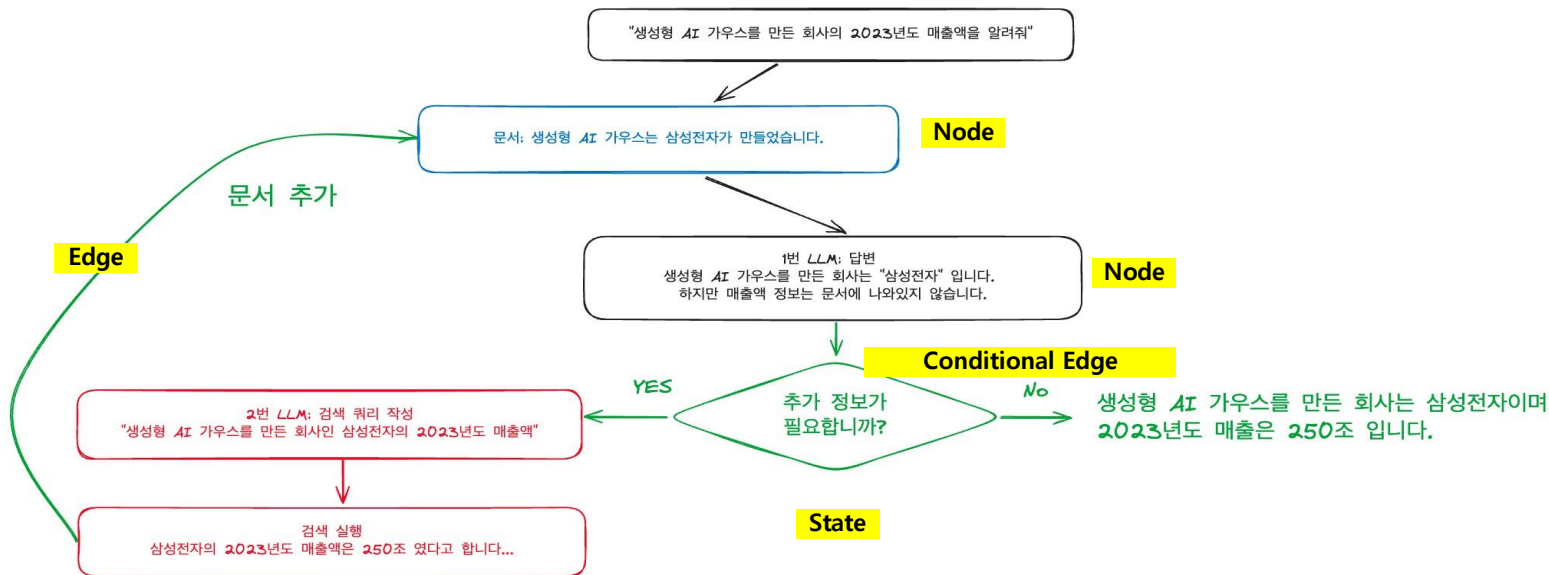


LangGraph



LangGraph로 구현하였을 시

- Node(노드): 어떤 작업(task) 을 수행할지 정의
- Edge(엣지): 다음으로 실행할 동작 정의
- State(상태): 현재의 상태 값을 저장 및 전달하는데 활용
- Conditional Edge(조건부 엣지): 조건에 따라 분기 처리

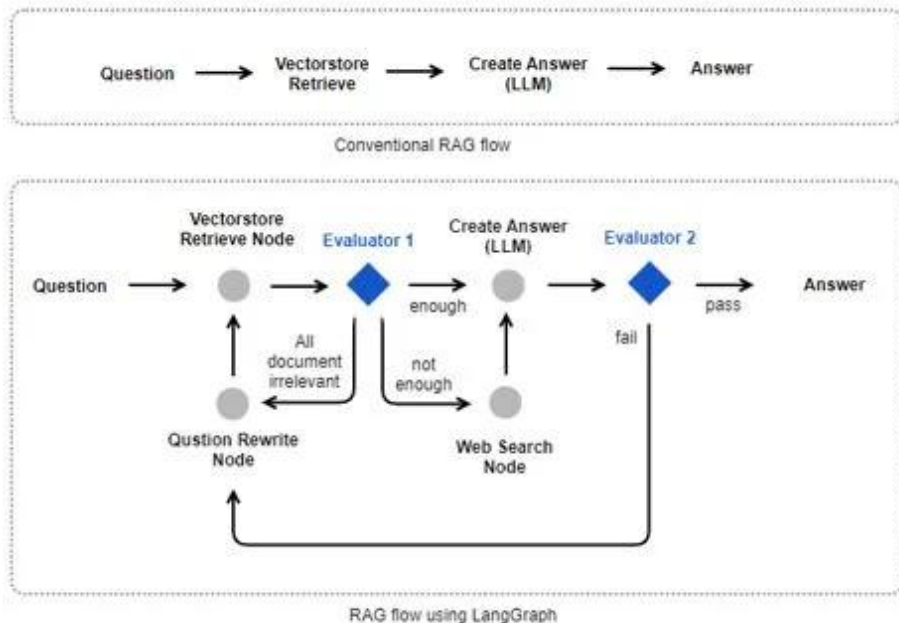


일반적인 RAG 구현과 LangGraph를 이용한 RAG 구현 (예)

여러 에이전트가 협력하여 복잡한 작업을 처리하거나, 특정 조건에 따라 작업 흐름을 분기하고 재처리하는 등의 구현

작업의 결과를 평가하고 이에 따라 기존 흐름이 재 처리되는 RAG를 구현하는 예

- 복잡한 에이전트 런타임 개발에 필수적인 순환 그래프를 만들고 관리할 수 있는 기능을 제공하고, 에이전트를 만드는데 필수적인 애플리케이션의 흐름과 상태를 세부적으로 제어
- 작업을 그래프 형태로 모델링 하여 각 작업의 단계를 노드(Node)와 간선(Edge)으로 관리하고 분기, 루프, 상태 관리 등을 통해 복잡한 작업 흐름을 체계적으로 처리



LangGraph

- **상태(State)**: 애플리케이션의 현재를 나타내는 데이터 구조로, 파이썬 TypedDict 또는 Pydantic 모델. 작업상태를 기록
- **노드(Node)**: 파이썬 함수로 정의되며, 상태(state)를 입력으로 LLM의 호출, 도구 실행 등 사용자가 정의한 작업을 실행하고 업데이트된 상태를 반환
- **간선(Edge)**: 마찬가지로 파이썬 함수로 정의되며, 특정 노드의 다음 실행 노드를 결정 / 간선에는 조건부 간선과, 일반 간선이 존재

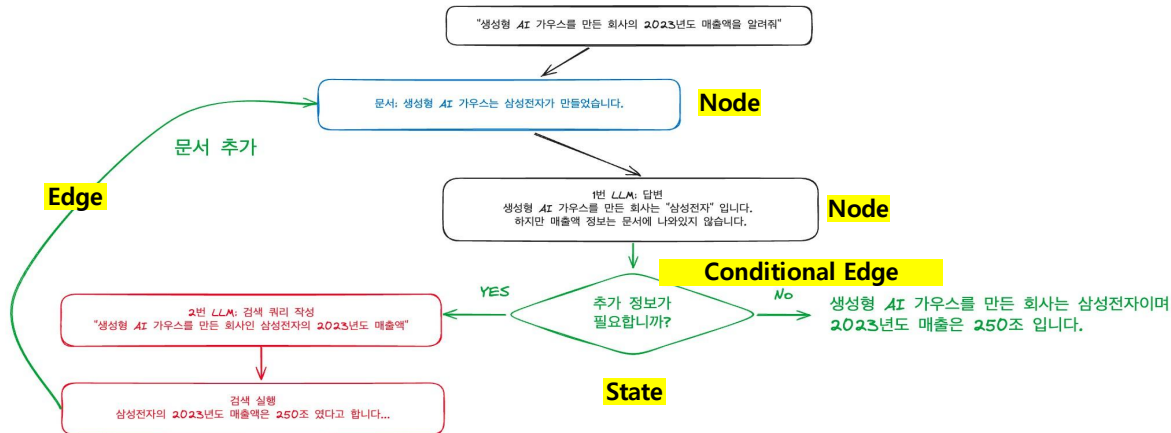
- Node(노드), Edge(엣지), State(상태관리) 를 통해 LLM 을 활용한 워크플로우에 순환(Cycle) 연산 기능을 추가하여 손쉽게 흐름을 제어

- RAG 파이프라인의 세부 단계별 흐름제어가 가능

- Conditional Edge: 조건부 (if, elif, else 와 같은..) 흐름 제어

- Human-in-the-loop: 필요시 중간 개입하여 다음 단계를 결정

- Checkpointer: 과거 실행 과정에 대한 “수정” & “리플레이” 기능



Workflow와 Agent를 구분해서 설계하기

LangGraph는 정해진 업무 흐름과 동적 도구 선택을 모두 구현할 수 있습니다.

구분	Workflow	Agent
제어 주체	개발자가 정의한 노드·엣지	LLM이 도구와 다음 행동 선택
예측 가능성	높음: 심사·승인·정형 프로세스	낮음: 탐색·조사·복합 질의
적합 업무	여신심사, 문서 검토, 배치 처리	리서치, 검색, 문제 해결, 도구 조합
핵심 설계	StateGraph + Conditional Edge	Tool Calling + ReAct Loop

판단 기준: 업무 순서가 명확하면 Workflow / 어떤 도구를 쓸지 LLM 판단이 필요하면 Agent

Node / Edge의 용어

Node (노드): 라틴어 **nodus**(마디, 매듭)에서 유래했습니다. 대나무의 마디나 신경계의 신경절처럼, 정보가 모이고 머무르는 '지점'을 의미합니다. 즉, 데이터가 저장되는 **주체(Entity)**가 됩니다.

Edge (엣지): 도형의 '**변(Side)**' 혹은 '**모서리**'를 의미합니다. 다각형에서 각 꼭짓점을 잇는 선을 'Edge'라고 부르는 것과 같습니다. 따라서 노드와 노드 사이를 잇는 '**관계(Relationship)**'를 의미하게 됩니다.

구분	개념	비유	예시 (SNS)
Node (Vertex)	데이터의 주체	점, 섬, 도시	사람, 게시물
Edge (Link)	데이터 간의 연결	선, 다리, 도로	친구 추가, 좋아요

* 네트워크 토폴로지나 **Edge Computing**(엣지 컴퓨팅)의 개념과는 다름

LangGraph / Graph DB 관점 용어

Graph DB (데이터 저장의 관점)

데이터베이스에서는 "누가 무엇을 했다"는 정보를 저장해야 합니다.

- **Node**: "사용자 A", "상품 B"와 같은 **명사적 객체**입니다.
- **Edge**: "구매했다", "좋아한다"와 같은 **동사적 관계**입니다. 만약 Edge가 객체라면, "구매했다"라는 동작 자체가 독립적으로 존재하기 어렵기 때문에 노드를 주인공으로 설정합니다.

LangGraph (워크플로우의 관점)

LangGraph는 상태 머신(State Machine)의 개념을 따릅니다.

- **Node**: 실제 계산이나 LLM 호출이 일어나는 **작업 단위(Function/Step)**입니다.
- **Edge**: 작업 A가 끝난 후 작업 B로 이동하는 **흐름(Control Flow)**입니다. 여기서 Node는 '일이 벌어지는 장소'이고, Edge는 '다음 칸으로 이동하는 길'이라고 이해하시면 쉽습니다.

StateGraph



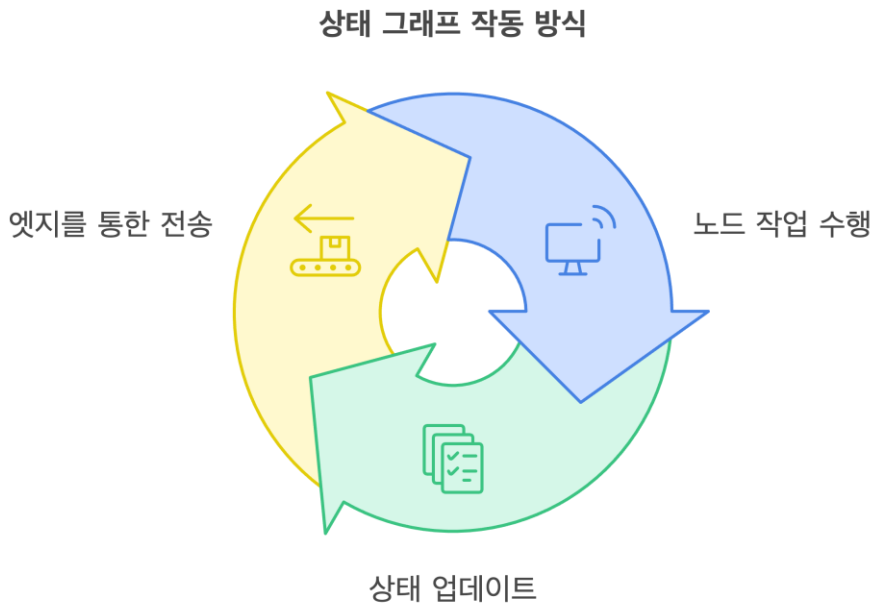
LangGraph – State (상태)

LangGraph의 핵심 구성 요소로, 상태 기반의 워크플로우를 구축하는 데 사용되는 가장 기본적인 그래프 구조이 그래프는 노드와 엣지로 구성되며, 각 노드는 특정 작업을 수행하고 상태를 업데이트

- 모든 노드가 같은 State를 보고 수정할 수 있습니다
- Python의 `TypedDict`를 사용해서 정의합니다
- 어떤 종류의 데이터든 저장할 수 있습니다

```
from typing import TypedDict

# GraphState 요소는 상태를 전달하는 용도로 사용됩니다.
class GraphState(TypedDict):
    goal: str # 목표
    todo: list[str] # 할 일 목록
    current_job: str # 현재 작업
    total_time: int # 총 소요시간(시간)
    time_spent: int # 소요 시간(시간)
    status: str # 상태(진행중, 다음 작업, 종료)
```



LangGraph – State (상태)

❖ 노드 별 상태 값의 변화

Key	Value
time	1
name	
llm	GPT

노드1

Key	Value
time	2
name	홍길동
llm	GPT

노드2

Key	Value
time	3
name	홍길동
llm	GPT

노드3

Key	Value
time	4
name	설리
llm	GPT

노드4

- 각 노드에서 새롭게 업데이트 하는 값은 기존 Key 값을 덮어쓰는 방식
- 노드에서 필요한 상태 값을 조회하여 동작에 활용할 수 있음
- 노드4 에서 노드1에서 반영된 llm 값이 그대로 상태 전달되어 조회가 가능

대화형 Agent에서는 MessagesState를 우선 고려

일반 리스트 누적보다 메시지 병합 규칙이 포함된 State를 쓰면 대화형 에이전트 구현이 단순해집니다.

일반 State

```
class State(TypedDict):  
    messages: list  
    context: list  
    answer: str
```

MessagesState

```
from langgraph.graph import MessagesState  
  
class AgentState(MessagesState):  
    context: list  
    answer: str
```

- Human / AI / Tool 메시지를 일관된 구조로 관리
- add_messages Reducer로 대화 이력 누적
- Tool Calling, ReAct, HITL 흐름에서 특히 유용
- RAG 문서 context는 별도 Key로 분리하는 것이 안전

LangGraph – Node (노드)

Node는 실제 작업을 수행하는 Python 함수 / 현재 상태를 받아서 새로운 상태를 돌려줌

- 항상 state를 첫 번째 매개변수
- 딕셔너리 형태로 새로운 상태를 반환
- 반환하지 않은 필드는 기존 값을 유지
- 함수로 정의
- 입력인자: 상태(State) 객체
- 반환(return)
 - 대부분 상태(State) 객체
 - Conditional Edge 의 경우 다를 수 있음

```
# 문서에서 검색하여 관련성 있는 문서를 찾습니다.  
def retrieve_document(state: GraphState) -> GraphState:  
    # Question에 대한 문서 검색을 retriever로 수행합니다.  
    retrieved_docs = pdf_retriever.invoke(state["question"])  
    # 검색된 문서를 context 키에 저장합니다.  
    return GraphState(context=format_docs(retrieved_docs))
```

Key	Value
context	
question	질문1
answer	
score	



동작을 위한 코드 구현

- Python 코드
- DB 조회
- API 호출
- ...



Key	Value
context	문서1
question	질문1
answer	
score	

입력

출력

LangGraph – 그래프 생성 후 Node 추가

```
from langgraph.graph import END, StateGraph
from langgraph.checkpoint.memory import MemorySaver
```

```
# langgraph.graph에서 StateGraph와 END를 가져옵니다.
```

```
workflow = StateGraph(GraphState)
```

```
# 노드들을 정의합니다.
```

```
workflow.add_node("retrieve", retrieve_document) # 에이전트 노드를 추가합니다.
```

```
workflow.add_node("llm_answer", llm_answer) # 정보 검색 노드를 추가합니다.
```

- 이전에 정의한 함수를 Graph 에 추가
- add_node(“노드이름”, 함수)

노드

```
# 문서에서 검색하여 관련성 있는 문서를 찾습니다.
```

```
def retrieve_document(state: GraphState) -> GraphState:
```

```
    # Question 에 대한 문서 검색을 retriever 로 수행합니다.
```

```
    retrieved_docs = pdf_retriever.invoke(state["question"])
```

```
    # 검색된 문서를 context 키에 저장합니다.
```

```
    return GraphState(context=format_docs(retrieved_docs))
```

노드

```
# Chain을 사용하여 답변을 생성합니다.
```

```
def llm_answer(state: GraphState) -> GraphState:
```

```
    return GraphState(
```

```
        answer=pdf_chain.invoke({"question": state["question"], "context": state["context"]}))
```

LangGraph – Edge(엣지)

Edge는 노드들 사이의 연결을 정의합니다. "이 작업 다음에 저 작업을 하자"라고 알려주는 역할

- 노드에서 노드간의 연결
- `add_edge("노드이름", "노드이름")`
from > to

```
# 각 노드들을 연결합니다.  
workflow.add_edge("retrieve", "llm_answer") # 검색 -> 답변  
workflow.add_edge("llm_answer", "relevance_check") # 답변 -> 관련성 체크
```



LangGraph – 조건부 Edge(엣지)

- 노드에 조건부 엣지를 추가하여 분기를 수행할 수 있습니다.
- `add_conditional_edges`(“노드이름”, 조건부 판단 함수, **dict** 로 다음 단계 결정) 흐름
- **“relevance_check”** 노드에서 나온 결과를 **is_relevant** 함수에 입력
- 반환된 값은 **“grounded”**, **“notGrounded”**, **“notSure”** 중 하나
 - value 에 해당하는 값이 END 면 Graph 실행 종료
 - **“llm_answer”** 와 같이 노드이름이면 해당 노드로 연결

```
# 조건부 엣지를 추가합니다.
workflow.add_conditional_edges(
    "relevance_check", # 관련성 체크 노드에서 나온 결과를 is_relevant 함수에 전달합니다.
    is_relevant,
    {
        "grounded": END, # 관련성이 있으면 종료합니다.
        "notGrounded": "llm_answer", # 관련성이 없으면 다시 답변을 생성합니다.
        "notSure": "llm_answer", # 관련성 체크 결과가 모호하다면 다시 답변을 생성합니다.
    },
)
```

State는 단순 Dictionary가 아니다

Reducer를 설계해야 상태 누적과 병렬 병합이 안정적으로 동작

- Reducer가 없으면 새 값으로 교체
- 리스트·로그·문서 결과는 누적이 필요한 경우가 많음
- 병렬 노드가 같은 Key를 수정하면 병합 규칙이 필요
- Node는 전체 State가 아니라 변경분만 반환

```
import operator
from typing import Annotated
from typing_extensions import TypedDict

class AgentState(TypedDict):
    question: str
    documents: Annotated[list, operator.add]
    logs: Annotated[list, operator.add]
    answer: str
```

Command: 상태 업데이트와 다음 노드 이동을 함께 처리

Conditional Edge가 흐름만 결정한다면, Command는 상태 변경과 이동을 한 번에 표현합니다.

```
from typing import Literal
from langgraph.types import Command

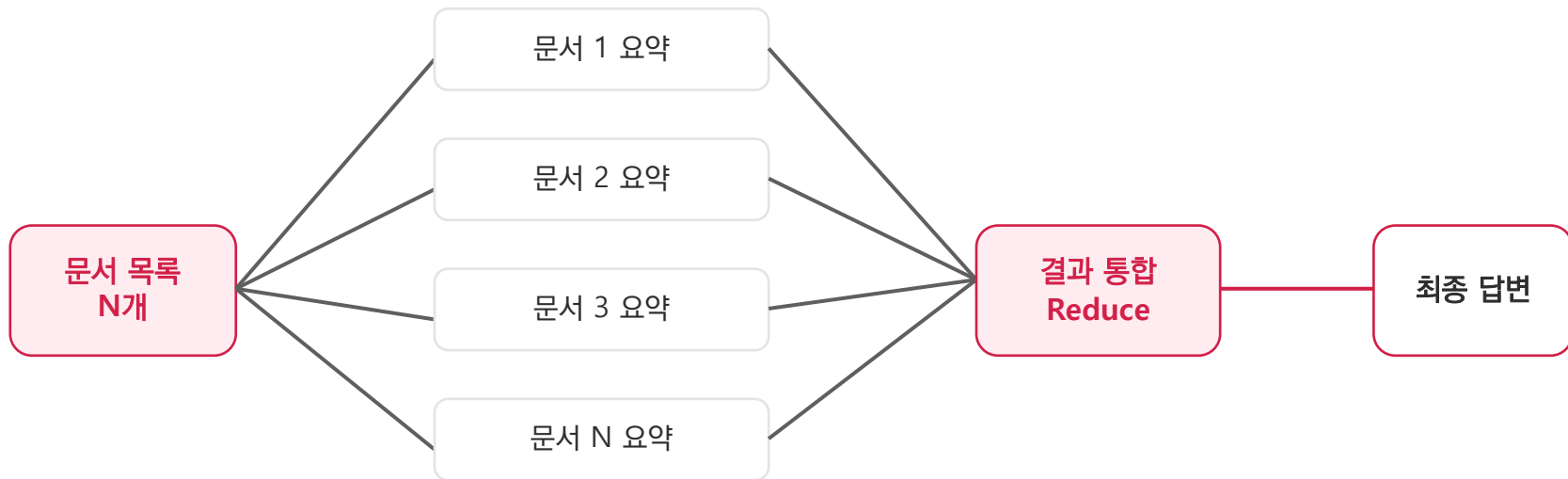
def evaluate(state) -> Command[Literal["generate", "retrieve"]]:
    if state["score"] >= 0.8:
        return Command(update={"status": "approved"}, goto="generate")
    return Command(update={"status": "retry"}, goto="retrieve")
```

방식	역할
Conditional Edge	별도 라우팅 함수가 다음 노드를 결정
Command	노드 내부에서 State update + goto 처리
사용처	Handoff, Multi-Agent, HITL 재개 흐름

평가 노드가 “결과 판단 + 다음 이동 + 상태 기록”을 동시에 책임질 때 유용

Send: 실행 중 동적으로 병렬 작업 만들기

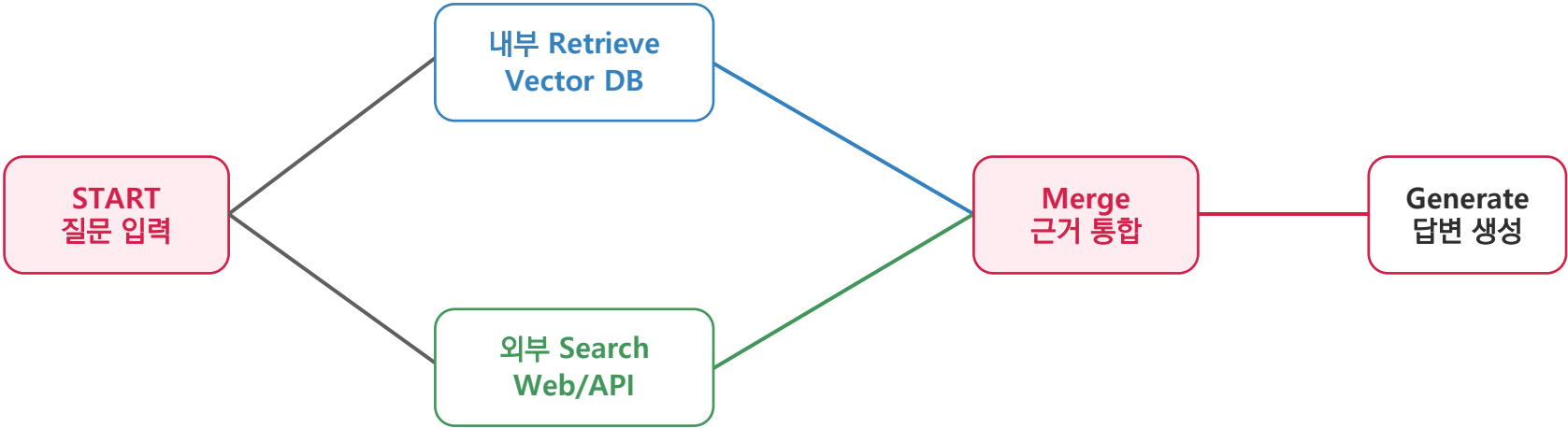
입력 개수가 실행 시점에 달라지는 Map-Reduce 형태의 작업에 적합합니다.



- 문서 수, 고객 수, 파일 수처럼 실행 전에는 작업 개수를 모를 때 사용
- 동적 병렬 처리 후 Reducer로 결과를 합치는 구조와 함께 설계

병렬 실행과 Superstep

여러 노드가 같은 단계에서 동시에 실행되고, 다음 단계에서 결과가 병합됩니다.



체크포인트	설명
Superstep 1	START 실행
Superstep 2	Retrieve와 Search 병렬 실행
Superstep 3	Merge에서 결과 병합

LangGraph – 시작점 지정

- `set_entry_point("노드이름")`
- 지정한 시작점부터 Graph 가 시작

```
# 시작점을 설정합니다.  
workflow.set_entry_point("retrieve")
```



그래프 실행

- RunnableConfig

- recursion_limit: 최대 노드 실행 개수를 지정합니다. (13인 경우: 총 13개의 노드까지 실행합니다)
- thread_id: 그래프 실행 아이디를 기록하고, 추후 추적하기 위한 목적으로 활용합니다.

- 상태(State) 로 시작합니다.

- 여기서 “question” 에 질문만 입력하고 상태를 첫 번째 노드에게 전달 합니다.

- invoke(상태, config) 전달하여 실행합니다.

```
from langchain_core.runnables import RunnableConfig

# recursion_limit: 최대 반복 횟수, thread_id: 실행 ID (구분용)
config = RunnableConfig(recursion_limit=13, configurable={"thread_id": "SELF-RAG"})

# GraphState 객체를 활용 (function) question: Any
inputs = GraphState(question="삼성전자가 개발한 생성형 AI의 이름은?")
output = app.invoke(inputs, config=config)

# 출력 결과를 확인합니다.
print("Question: \t", output["question"])
print("Answer: \t", output["answer"])
print("Relevance: \t", output["relevance"])
```

결과 확인

- 출력된 결과로 최종 확인합니다.
- 출력 결과 역시 상태(State)에 담겨 있습니다.

```
# 출력 결과를 확인합니다.  
print("Question: \t", output["question"])  
print("Answer: \t", output["answer"])  
print("Relevance: \t", output["relevance"])
```



```
Question:      삼성전자가 개발한 생성형 AI의 이름은?  
Answer:       삼성전자가 개발한 생성형 AI의 이름은 '삼성 가우스'입니다. (출처: data/SPRI_AI_Brief_2023년12월호_F.pdf, 페이지 13)  
Relevance:    grounded
```

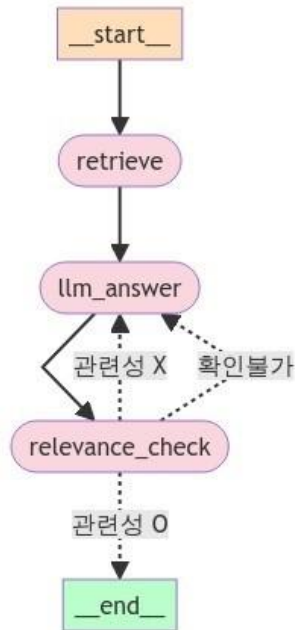
그래프 시각화

노드와 간선으로 인해 그래프가 더욱 복잡해지면 그래프를 시각화하기 어려울 수 있지만, LangGraph에는 그래프를 시각화하는 여러 가지 방법이 내장되어 있다. 이를 통해 정의된 그래프를 한 눈에 시각화 할 수 있다.

- `get_graph(xray=True).draw_mermaid_png()`

- 생성한 그래프 시각화

```
display(  
    Image(app.get_graph(xray=True).draw_mermaid_png())  
) # 실행 가능한 객체의 그래프를 mermaid 형식의 PNG로 그려서 표시합니다.
```



그래프 시각화 다음에는 Runtime Trace가 필요

Mermaid는 구조를 보여주고, Trace는 실제 실행에서 무엇이 일어났는지 보여줍니다.

Mermaid / Graph View

- 노드·엣지 구조 확인
- 분기 가능 경로 파악
- 강의/문서화에 적합

Runtime Trace

- 실제로 실행된 노드 순서
- 각 노드의 입력·출력 State
- LLM/Tool 호출 시간과 비용
- 오류 발생 지점 및 재시도 내역

정적 구조 확인에서 끝내지 말고
실행 로그 기반으로 디버깅·평가까지 연결

테스트 전략: Node / Routing / Graph 분리

LangGraph는 그래프 전체를 한 번에 검증하기보다 작은 단위부터 테스트하는 것이 안정적입니다.

1. Node 단위 테스트

- 입력 State → 출력 Update 확인
- LLM Mock으로 결정성 확보
- 도구 실패 케이스 분리

2. Routing 테스트

- score, status에 따른 분기 확인
- 무한 루프 방지 조건 검증
- END 조건 테스트

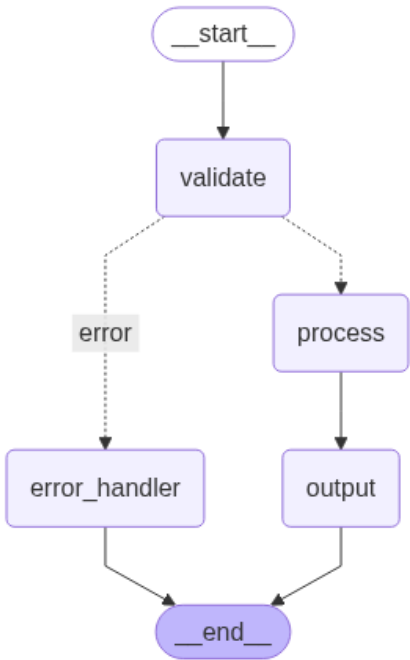
3. Graph 통합 테스트

- 정상 경로와 실패 경로 실행
- thread_id로 상태 유지 확인
- HITL 승인 전 실행 차단

```
def test_route_by_score():  
    assert route({"score": 0.9}) == "generate"  
    assert route({"score": 0.2}) == "retrieve"
```

그래프 구성요소 연결 및 실행

LangGraph 애플리케이션의 **아키텍처 품질**을 결정하는 핵심 과정입니다. 그래프 구성은 단순히 노드와 엣지를 연결하는 것을 넘어서, **비즈니스 로직의 흐름을 명확하게 표현**하고 **예외 상황을 적절하게 처리**하며 **시스템의 안정성과 확장성**을 보장



실행 방법 개요

메서드	설명	용도
<code>invoke()</code>	동기 실행, 최종 결과 반환	배치 처리, 테스트
<code>stream()</code>	동기 스트리밍, 중간 결과 반환	실시간 UI
<code>ainvoke()</code>	비동기 실행	고동시성 서버
<code>astream()</code>	비동기 스트리밍	비동기 실시간 UI

Memory



기억 | Memory

메모리(Memory) 기능은 그래프의 상태를 저장하고 복원하는 핵심 메커니즘 이를 통해 1) 대화의 맥락을 유지하고, 2) 오류 발생 시 안전하게 복구하며, 3) 사용자별로 개인화된 경험을 제공할 수 있음

일상 대화에서 우리는 자연스럽게 이전 내용을 기억

나: "오늘 점심으로 피자를 먹었어"
친구: "맛있었어?"
나: "응, 정말 맛있었어!"
친구: "어떤 피자?" ← 친구는 "피자"를 기억하고 있어요!

AI 애플리케이션도 마찬가지입니다.
이전 대화나 작업 상태를 기억해야 자연스러운 상호작용이 가능

장점	설명	예시
자연스러운 대화	이전 내용을 참고해서 대답	"아까 말한 그 책 제목이 뭐였죠?"
개인화	사용자마다 다른 정보 기억	Alice와 Bob에게 각각 다른 응답
안전한 실행	오류 발생 시 복구 가능	5단계에서 실패 → 5단계부터 재시작

메모리가 없으면 어떻게 될까?

메모리가 없는 AI

사용자: "내 이름은 김철수야"

AI: "안녕하세요, 김철수님!"

사용자: "내 이름이 뭐였지?"

AI: "죄송합니다, 저는 이전 대화를 기억하지 못합니다." # 😞

메모리가 있는 AI

사용자: "내 이름은 김철수야"

AI: "안녕하세요, 김철수님!"

사용자: "내 이름이 뭐였지?"

AI: "김철수님이라고 하셨습니다!" # 😊 이전 대화를 기억!

Memory System

LangGraph는 단기 메모리와 장기 메모리를 분리하여 2단계로 관리

메모리 유형	범위	구현	특징
단기 메모리	Thread-scoped (단일 대화)	Checkpointter	대화 히스토리 유지, 세션 종료 시 정리 가능
장기 메모리	User/App-scoped (스레드 간 공유)	Store	사용자 프로필/선호도, 영구 저장, 시맨틱 검색 가능

단기 메모리 (Checkpointing)

체크포인트는 게임의 세이브 포인트와 같습니다. 그래프 실행의 특정 시점에서 상태를 저장하여 나중에 그 지점부터 다시 시작할 수 있게 해줍니다.

기능	설명
대화 연속성	동일한 thread_id로 이전 대화 맥락 유지
오류 복구	실행 중 문제 발생 시 마지막 안정 상태로 복원
Human-in-the-Loop	특정 지점에서 인간의 개입과 승인 허용
Time Travel	과거 체크포인트로 돌아가 다른 경로 탐색

```
# 기록을 위한 메모리 저장소를 설정합니다.  
memory = MemorySaver()  
  
# 그래프를 컴파일합니다.  
app = workflow.compile(checkpointer=memory)
```

thread_id와 checkpoint_id 구조

Thread는 실행 흐름을 묶고, Checkpoint는 그 안의 특정 상태 시점을 가리킵니다.

thread_id = user_alice_session_01

checkpoint_id = cp_001

checkpoint_id = cp_002

checkpoint_id = cp_003

checkpoint_id = cp_004

용어	의미
thread_id	하나의 대화·실행 단위를 묶는 식별자
checkpoint_id	Thread 내부의 특정 상태 저장 지점
같은 thread_id	이전 상태를 이어받아 실행 가능
과거 checkpoint	Time Travel 또는 분기 실행의 기준점

핵심: checkpoint_id가 계속 생기는 것은 정상이며, 기억 단위는 thread_id로 이해하는 것이 쉽습니다.

장기 메모리 (Store)

Store는 스레드 간 공유 가능한 Key-Value 저장소. **Namespace**로 데이터를 구조화하며, **index** 설정으로 시맨틱 검색 지원

```
from langgraph.store.memory import InMemoryStore

store = InMemoryStore(
    index={
        "embed": embeddings, # 임베딩 모델
        "dims": 1536,        # 임베딩 차원
        "fields": ["text"],  # 임베딩할 필드
    }
)

# Namespace 기반 저장
store.put(("user_123", "preferences"), "food", {"text": "매운 음식을 좋아합니다"})

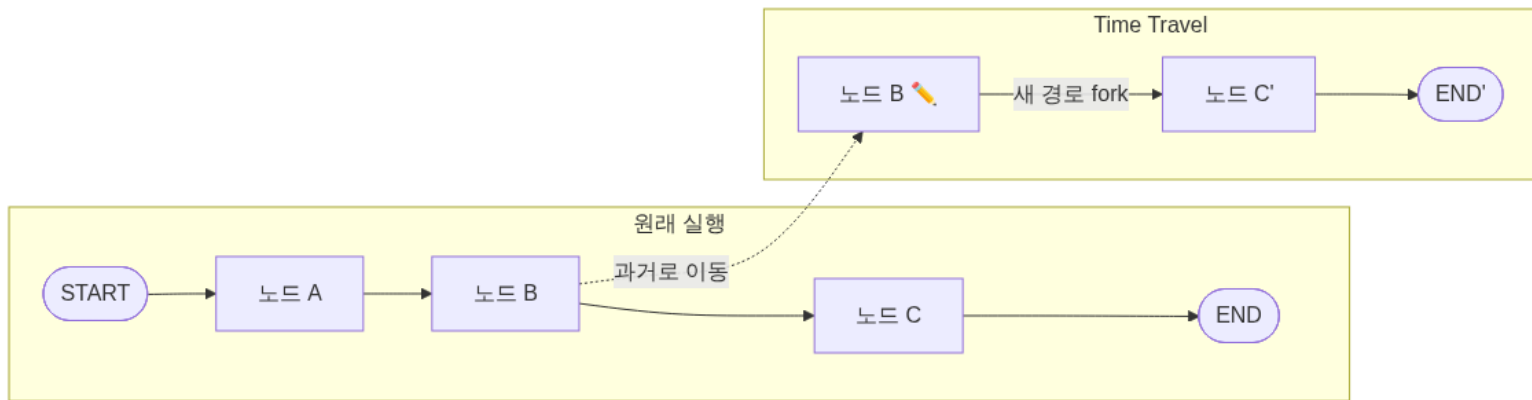
# 시맨틱 검색
results = store.search(("user_123", "preferences"), query="음식 취향", limit=3)
```

체크포인트	저장 위치	용도	특징
InMemorySaver	메모리	개발/테스트	프로세스 종료 시 소멸
SqliteSaver	SQLite DB	로컬 영구 저장	파일 기반, 단일 노드
PostgresSaver	PostgreSQL	프로덕션 권장	분산 환경,고가용성

선택가이드
빠른 설정, 간편한 사용 (개발/테스트)
파일 기반 영구 저장 (로컬 프로덕션)
고가용성, 다중 인스턴스 (서버 프로덕션)

Time Travel - 과거 상태로 이동

그래프 실행의 **과거 체크포인트로 돌아가** 다른 경로를 탐색할 수 있게 해주는 강력한 디버깅 및 탐색 기능 / Git에서 이전 커밋으로 체크아웃하듯이, 특정 시점의 상태로 되돌아가 대안적인 실행 경로를 시도할 수 있음



AI 에이전트는 종종 **비결정적(Non-deterministic)** 특성을 가지므로, 동일한 입력에 대해 다른 결과를 생성할 수 있음

추론 과정 이해 (Understand Reasoning)

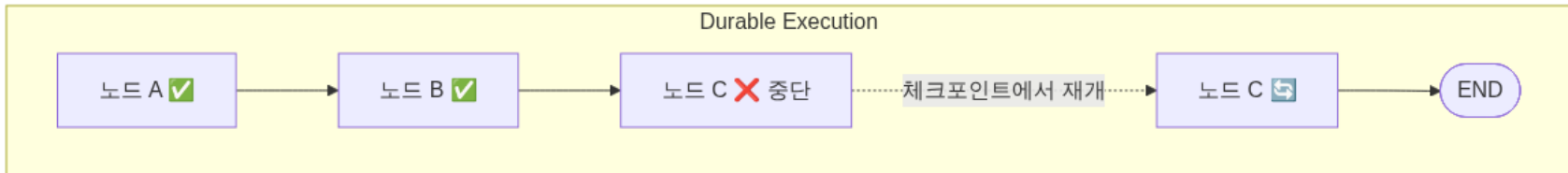
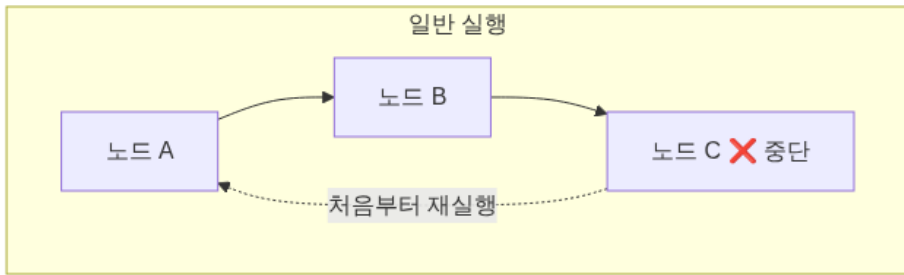
오류 디버깅 (Debug Mistakes)

대안 탐색 (Explore Alternatives)

Durable Execution – 견고한 실행

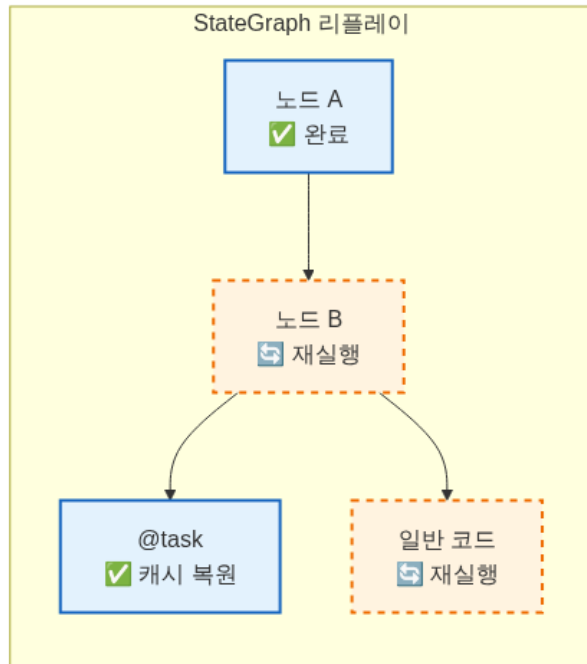
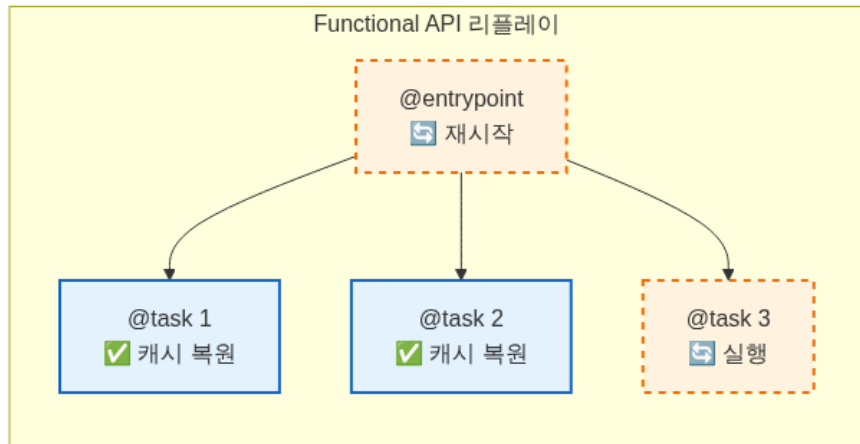
LangGraph의 **중단 내성(Fault Tolerance)** 기능으로, 그래프 실행이 중단되더라도 **진행 상태를 보존**하여 나중에 정확히 그 지점부터 재개할 수 있게 해줍니다.

마치 장시간 게임을 플레이할 때 자동 저장 기능이 있는 것처럼, 실행 과정이 자동으로 저장되어 언제든지 복구할 수 있음



Durable Execution – 리플레이 메커니즘

워크플로우 재개 시 LangGraph는 정확히 멈춘 코드 줄에서 다시 시작하지 않음. 대신 적절한 시작점에서 리플레이



Time Travel과 Replay 주의사항

과거 상태로 돌아간다고 해서 코드의 특정 줄에서 그대로 이어지는 것은 아닙니다.



- 외부 부수 효과가 있는 노드(이메일 전송, DB 수정, 결제)는 재실행 주의
- 중복 실행을 막기 위해 실행 기록, idempotency key, 승인 노드 설계 필요
- Time Travel은 디버깅과 대안 탐색에 강하지만 운영 환경에서는 정책이 필요

오류 처리: Retry, Timeout, 실패 노드

실무 그래프는 성공 경로보다 실패 경로를 명확히 설계해야 안정적으로 운영됩니다.

오류 유형	대응 방식	State에 남길 정보
네트워크 / Rate Limit	RetryPolicy, Backoff	attempt_count, last_error
도구 입력 오류	입력 재생성 또는 사용자 확인	tool_name, invalid_args
검색 결과 부족	Query Rewrite, Web Search	retrieval_score, fallback_used
재시도 초과	실패 노드 또는 HITL 이동	failed_reason, next_action

```
builder.add_node(  
    "web_search",  
    web_search,  
    retry_policy=RetryPolicy(max_attempts=3)  
)
```

메모리 기본 설정

```
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver # 1 체크포인트어 import

# ... 그래프 정의 ...

memory = InMemorySaver() # 2 체크포인트어 생성
app = graph.compile(checkpointer=memory) # 3 그래프에 연결

# 실행 시 thread_id로 세션 구분
config = {"configurable": {"thread_id": "user_123"}}
app.invoke(initial_state, config=config)
```

thread_id란?

`thread_id` 는 대화 세션을 구분하는 고유 식별자입니다.

- 같은 `thread_id` → 같은 대화 기록 공유
- 다른 `thread_id` → 완전히 별개의 대화

실제 서비스에서는 사용자 ID, 채팅방 ID 등을 `thread_id` 로 사용합니다.

메모리 상태 확인하기

저장된 상태를 직접 확인하고 싶을 때 `get_state()` 메서드를 사용합니다.

```
# 현재 상태 가져오기
state = app.get_state(config)

# 저장된 값 확인
print(state.values)      # 실제 데이터
print(state.config)      # 설정 정보
print(state.created_at)  # 생성 시간
print(state.parent_config) # 이전 상태 참조 (Time Travel에서 사용)
```

Sub-Graph / Streaming / Functional API

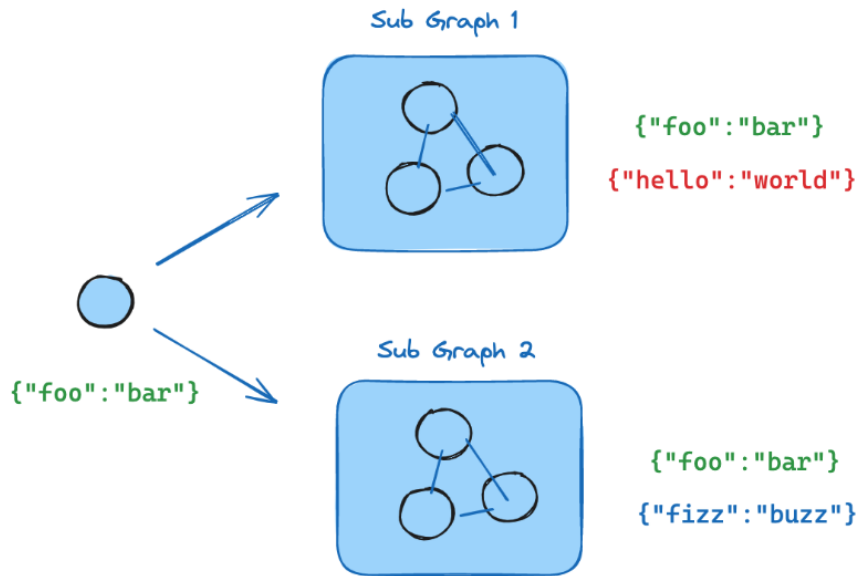


Sub-Graph

- LangGraph에서 복잡한 AI 시스템을 관리 가능한 단위로 모듈화하는 핵심 개념
- 더 큰 그래프 내에서 독립적으로 실행되는 작은 그래프로, 자체적인 상태와 노드, 엣지를 가질 수 있음

서브그래프의 활용 목적

- **멀티 에이전트 시스템**: 각 에이전트를 독립된 서브그래프로 구현
- **재사용**: 동일한 노드 집합을 여러 그래프에서 재사용
- **분산 개발**: 서브그래프 인터페이스 (입출력 스키마)만 준수하면 독립적으로 개발 가능



스트리밍 (Streaming)

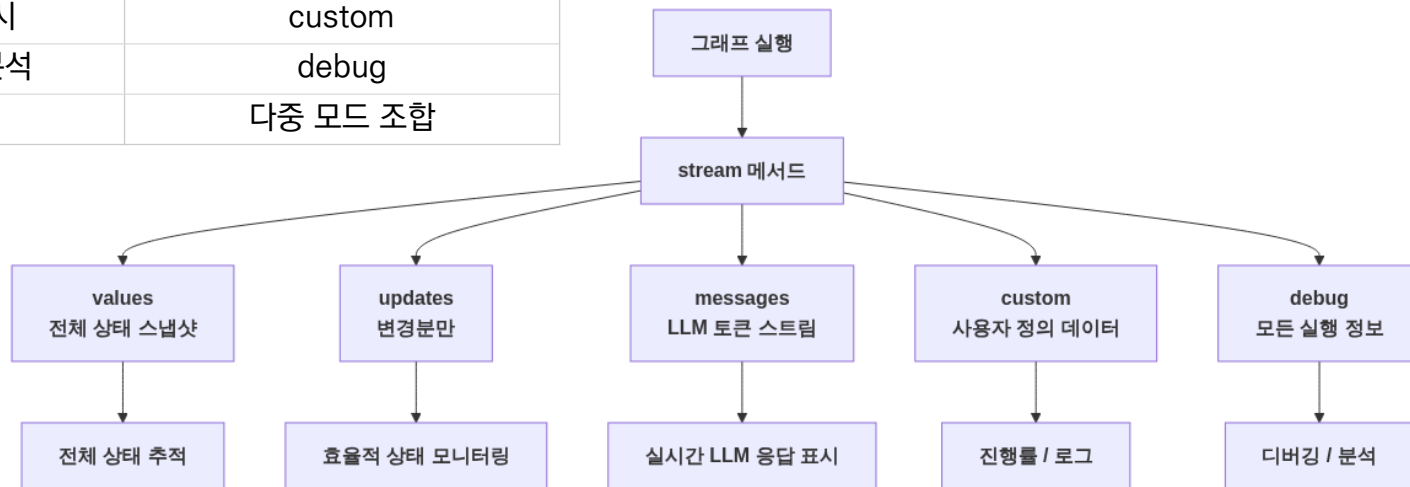
- 그래프 실행 과정에서 발생하는 다양한 이벤트와 데이터를 실시간으로 스트리밍할 수 있으며
- 여러 모드를 제공하여 원하는 정보만 선택적으로 받을 수 있음

Mode	설명	주요 용도
values	각 단계 후 전체 상태를 스트리밍	그래프 전체 상태 추적
updates	각 단계의 상태 변화분만 스트리밍	효율적인 상태 변경 추적
messages	LLM 토큰을 (message_chunk, metadata) 튜플로 스트리밍	실시간 LLM 응답 표시
custom	노드 내부에서 사용자 정의 데이터 스트리밍	진행 상황, 로그 등 커스텀 정보
debug	실행 과정의 모든 정보를 상세히 스트리밍	디버깅 및 상세 분석

LangGraph 애플리케이션을 개발할 때, 실행 중인 그래프의 상태를 실시간으로 모니터링하거나 LLM의 응답을 토큰 단위로 받아보고 싶은 경우가 많음
특히 사용자 경험을 향상시키기 위해서는 긴 작업이 진행되는 동안 중간 결과를 실시간으로 표시

스트리밍 모드 데이터 흐름

상황	권장 모드
UI에 전체 상태 표시	values
효율적인 변경 추적	updates
챗봇 실시간 응답	messages
진행률/로그 표시	custom
디버깅 및 문제 분석	debug
복합 정보 필요	다중 모드 조합



Functional API

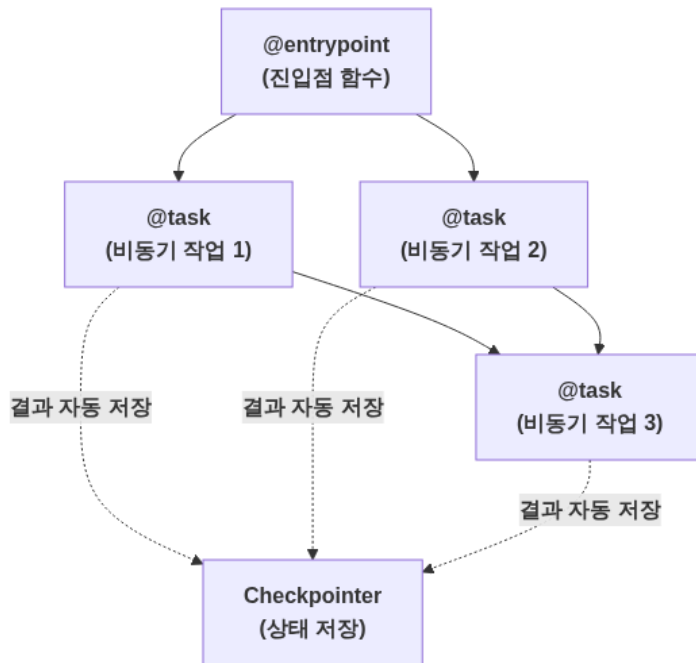
Functional API는 복잡한 그래프 구조 없이도 LangGraph의 핵심 기능을 활용할 수 있는 프로그래밍 인터페이스

- **@entrypoint**: 워크플로우의 진입점을 정의하는 데코레이터. Checkpointer와 연동하여 persistence와 human-in-the-loop를 지원합니다.
- **@task**: 개별 비동기 작업 단위를 정의하는 데코레이터. 각 task의 결과는 체크포인트에 자동 저장됩니다.

Functional API의 주요 특징

Functional API는 다음과 같은 상황에서 특히 유용합니다.

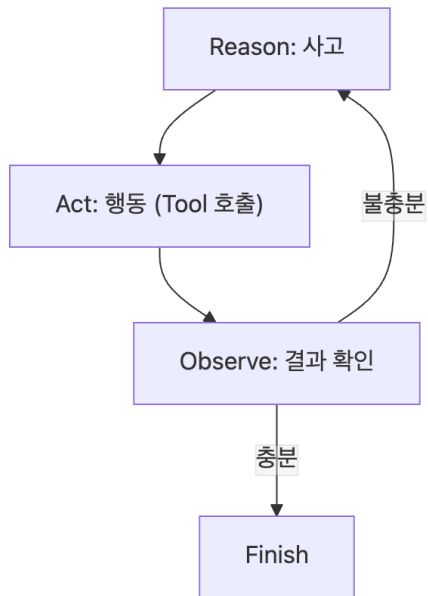
- 기존 코드에 LangGraph 기능 추가: 복잡한 리팩토링 없이 persistence, memory, human-in-the-loop 등의 기능을 점진적으로 통합
- 간단한 워크플로우 구현: 명시적인 그래프 구조가 필요하지 않은 선형적이거나 간단한 분기 로직
- Python 표준 구문 활용: if, for, 함수 호출 등 익숙한 제어 흐름 사용
- 빠른 프로토타이핑: 아이디어를 빠르게 검증하고 구현



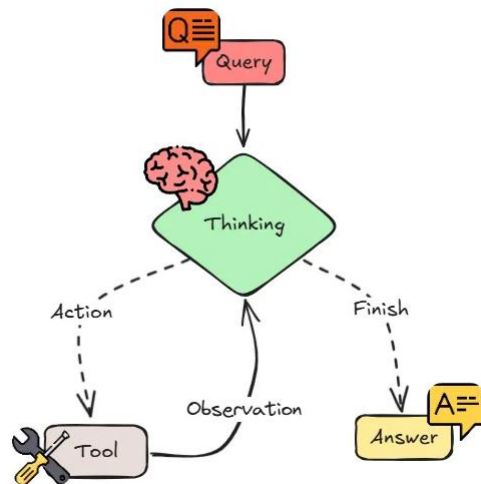
ReAct

ReAct

LLM이 도구를 사용하여 문제를 해결하는 방식을 체계화한 패러다임. 단순히 질문에 즉시 답하는 방식과 달리, ReAct 에이전트는 **추론(Thought) → 행동(Action) → 관찰(Observation)**의 순환 과정을 통해 복잡한 문제를 단계별로 분해하고 해결



ReAct Agent architecture



ReAct = **RE**ASONING(추론) + **ACT**ing(행동)의 줄임말입니다. 이 두 가지 핵심 요소가 상호작용하며 문제해결 능력을 극대화합니다.

ReAct vs 전통적 방식 상세 비교

- **ReAct 추천:** 실시간 정보 필요, 정확한 계산 필요, 다단계 추론, 답변 검증이 중요한 경우
- **전통적 방식 추천:** 일반 지식 질문, 창작, 번역 등 외부 도구가 불필요한 작업, 비용/속도가 중요한 경우

평가 항목	전통적 방식	ReAct 방식
정확도	내부 지식 한계	실시간 검증
계산 능력	추론 기반 (오류 가능)	도구 기반 (정확)
실시간 정보	불가능	Tavily 검색 활용
답변 검증	불가능	각 단계 추적 가능
비용	토큰 1회	토큰 3-5회
응답 시간	1-3초	5-15초

Chain-of-Thought vs ReAct

순수 추론 방식(Chain-of-Thought)과 ReAct의 핵심 차이는 외부 도구 활용 여부

항목	Chain-of-Thought	ReAct
추론 방식	내부 지식만 사용	추론 + 도구 호출
실시간 정보	불가	가능
환각 위험	높음	낮음
적합한 상황	단순 추론	복잡한 정보 수집

장점과 구현시 고려해야할 사항

ReAct의 장점

ReAct 패턴은 기존 방식 대비 다음과 같은 장점을 가집니다.

- **해석 가능성:** 각 추론 단계가 명시적으로 기록되어 의사결정 과정을 추적할 수 있습니다.
- **신뢰성:** 외부 도구로 정보를 검증하므로 환각(hallucination) 문제가 줄어듭니다.
- **확장성:** 새로운 도구를 추가하여 에이전트 능력을 손쉽게 확장할 수 있습니다.
- **제어 가능성:** 각 단계에서 제약 조건이나 인간 개입을 적용할 수 있습니다.

구현시 고려사항

구현 시 반드시 고려해야 할 사항은 다음과 같습니다.

- **무한 루프 방지:** 반복 횟수 제한(max_iterations) 설정이 필수입니다.
- **오류 처리:** 도구 호출 실패나 예외 상황에 대한 견고한 처리가 필요합니다.
- **컨텍스트 관리:** 긴 추론 과정에서 중요 정보가 손실되지 않도록 상태를 설계해야 합니다.

2025년 ReAct 발전 동향

- **멀티모달 ReAct**: 텍스트를 넘어 이미지, 음성, 비디오 처리
- **Self-Correcting ReAct**: 자체 검증과 오류 수정 기능
- **Collaborative ReAct**: 여러 전문 에이전트 간의 협력

멀티모달 ReAct의 핵심은 **다양한 입력을 통합하여 추론**하는 능력입니다.

- **이미지 분석**: 차트 해석, 객체 인식, OCR
- **음성 처리**: 텍스트 변환, 감정 분석
- **비디오 이해**: 프레임별 분석, 동작 인식

Self-Correcting ReAct (자기 수정 ReAct)

에이전트가 자신의 답변을 스스로 검증하고, 품질이 부족하면 재생성하는 패턴입니다. LangGraph **StateGraph**로 구현

Collaborative ReAct (협력적 ReAct)

여러 전문 에이전트가 역할을 분담하여 복잡한 문제를 협력 해결하는 패턴

핵심 구성 요소

구성 요소	설명	예시
ReasonNode	LLM이 문제를 분석하고, 어떤 도구를 쓸지 판단	“현재 날씨를 알려면 get_weather를 써야겠다.”
ToolNode	LLM이 선택한 도구를 실행	get_weather("Seoul")
ObserveNode	Tool 결과를 검토하고 판단	“결과가 이상하면 다시 시도.”
MemorySaver	상태 저장 및 루프 추적	이전 reasoning 및 tool 결과 누적

Self RAG & Corrective RAG

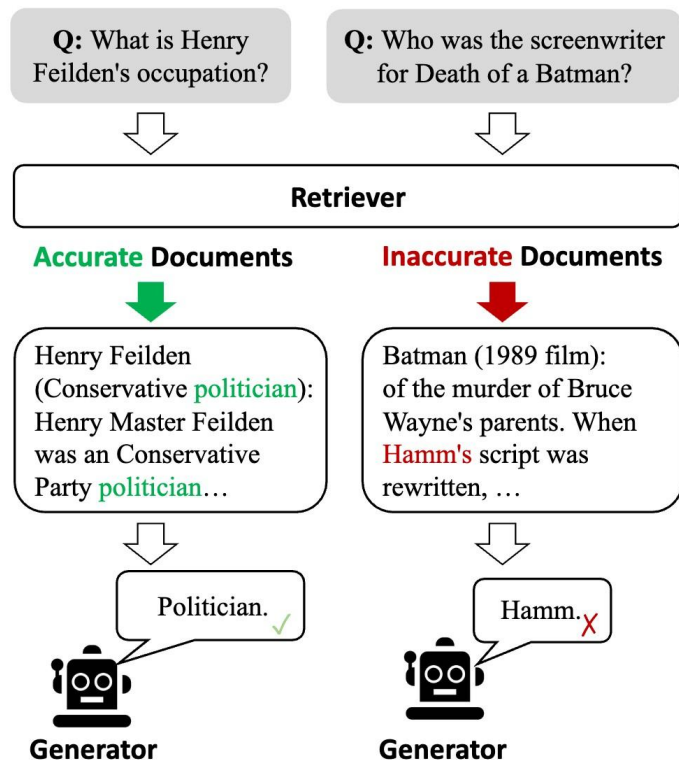


Corrective RAG [CRAG]

- 배경
 - RAG의 답변 결과는 검색된 문서의 관련성에 크게 의존적
 - 따라서, 검색이 잘못된 경우 답변에 대한 품질 우려가 됨
- 제안
 - 사용자 입력 쿼리(query)에 대하여 검색된 문서의 품질을 평가
 - 한마디로 검색된 결과가 사용자 입력 쿼리와 관련성이 높도록 쿼리를 수정(Corrective)

CRAG의 핵심은 "검색된 문서의 품질 보증"

검색 엔진(Retriever)이 가져온 문서가 질문과 관련이 없는 경우, 이를 교정(Corrective)하기 위해 웹 검색 등을 병행

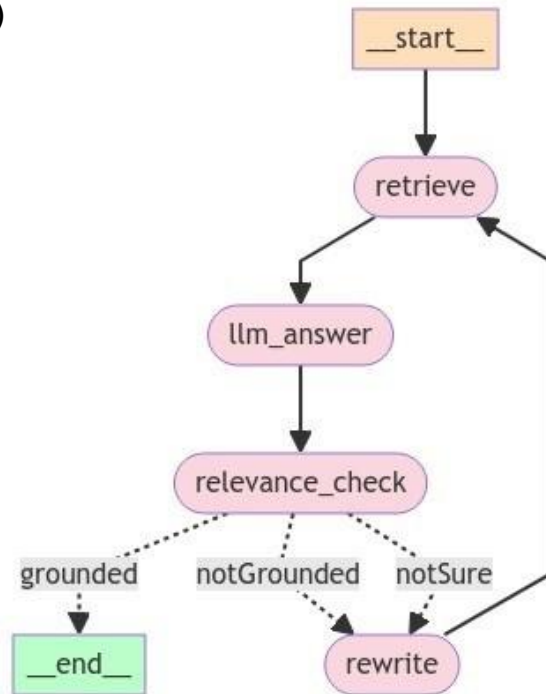


Corrective RAG

- 이전의 Relevance Check 를 수행 후 결과에 따라 쿼리를 조정 (Corrective)
- 재작성 된 쿼리로 다시 문서 검색 수행

주요 프로세스

1. **Retrieval**: 질문과 관련된 문서를 검색합니다.
2. **Relevance Check**: 평가자(Grader) 역할을 하는 LLM이 검색된 문서들이 질문에 정말 도움이 되는지 'Yes/No'로 판단합니다.
3. **Action Decision**:
 1. **Relevant**: 문서가 좋다면 바로 답변 생성을 진행합니다.
 2. **Irrelevant**: 문서가 나쁘다면, 기존 문서를 버리고 **웹 검색(Tavily 등)**을 통해 새로운 정보를 보충합니다.
 3. **Ambiguous**: 부분적으로 유효하다면 검색과 기존 문서를 혼합합니다.
4. **Generate**: 최종적으로 정제된 정보를 바탕으로 답변을 생성합니다.



Self-RAG

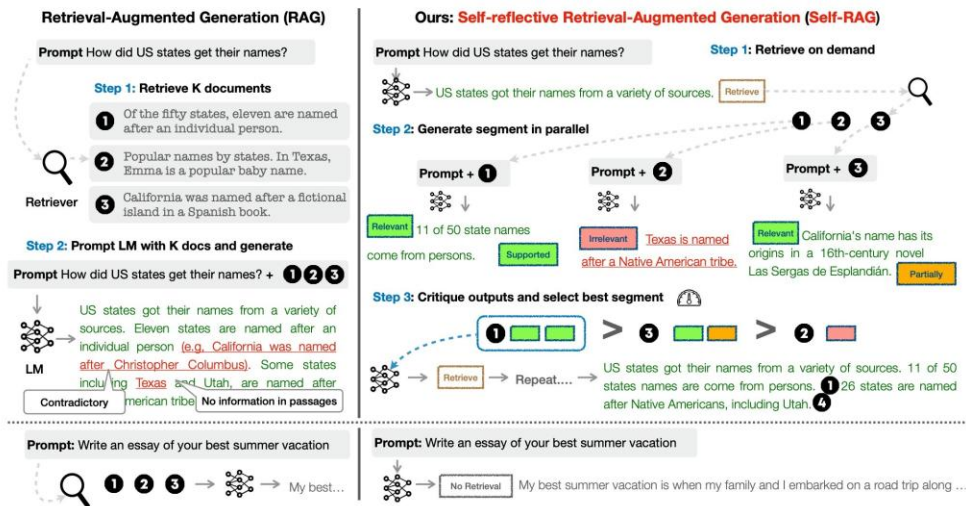
• 배경

- **Fixed Size Retrieval**: 좋은 싫은 정해진 크기만큼 검색하여 가져오기 때문에, 검색에 노이즈가 있을 수 있음
- 무분별하게 주입되는 검색으로 인하여 문서 내 다른 정보를 참고하거나, 제대로 된 답변이 나오지 않는 경우가 있음
- 검색된 정보의 정확성을 신뢰할 수 없음

• 제안

- 선택적 Retrieval 을 도입 (필요한 만큼만 Retrieve)
- Retrieval 로부터 답변을 도출
- 도출된 답변과 Retrieval Passage 간 관련성 체크

Self-RAG는 CRAG보다 한 단계 더 나아가 "답변의 품질과 환각 유무"까지 스스로 체크합니다. 단순히 문서만 보는 게 아니라, 생성된 답변이 문서에 기반했는지(Grounding), 질문에 대한 최종 답변으로 적절한지를 다각도로 평가



Self-RAG : 관련성 체크 흐름

- 코드 한 줄로 흐름 제어
 - `relevance_check > llm_answer`
 - `relevance_check > retrieve`

주요 프로세스

1.Retrieve: 관련 문서를 검색합니다.

2.Grade Documents: 문서가 질문과 관련 있는지 평가합니다 (CRAG와 유사).

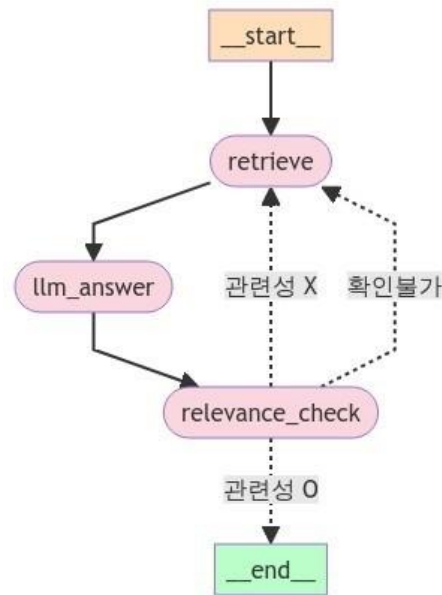
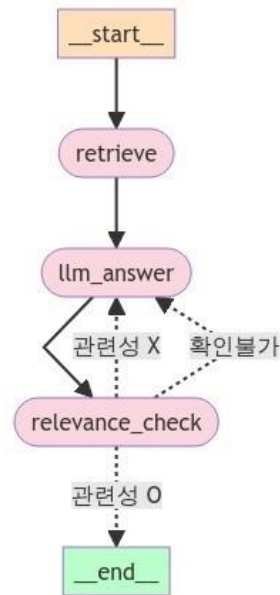
3.Generate: 관련 있는 문서만을 사용하여 답변을 생성합니다.

4.Self-Grade (Hallucination Check): 생성된 답변이 검색된 문서의 내용 안에 포함되는지(Support) 확인합니다.

1. 만약 문서에 없는 내용을 지어냈다면(환각), 다시 생성 노드로 돌아갑니다.

5.Self-Grade (Utility Check): 답변이 사용자의 질문에 대한 실제 해결책이 되는지 평가합니다.

1. 도움이 되지 않는다면 다시 검색(Retrieval) 단계부터 시작하거나 다른 경로를 탐색합니다.



핵심 차이점

구분	Corrective RAG (CRAG)	Self-RAG
주요 목적	검색 결과의 정확성 보완	생성된 답변의 신뢰성(환각 방지) 및 유용성 검증
실패 시 대응	웹 검색 등 외부 도구를 활용해 지식 보충	루프(Iteration)를 통해 답변 재생성 또는 재검색
판단 시점	답변 생성 전 (문서 평가)	답변 생성 전(문서 평가) + 생성 후(답변 평가)
복잡도	상대적으로 단순함	여러 단계의 평가 노드가 필요하여 더 복잡함

이러한 패턴을 사용하면 RAG 시스템의 신뢰도를 획기적으로 높일 수 있으며,
특히 전문적인 지식이 필요한 도메인에서 오답을 줄이는 데 매우 효과적

Guardrail은 입력·실행·출력 단계로 분리

Self-RAG나 CRAG가 있어도 보안과 규정 준수는 별도 설계가 필요합니다.

입력 단계

- Prompt Injection 탐지
- 파일 형식·크기 제한
- 개인정보 마스킹

실행 단계

- 허용된 Tool만 호출
- SQL Read/Write 권한 분리
- 비용·반복 횟수 제한

출력 단계

- 출처 없는 답변 제한
- 민감정보 검증
- 근거 문서와 답변 비교

고위험 작업은 규칙 기반 검증 + Human-in-the-Loop 승인을 함께 적용

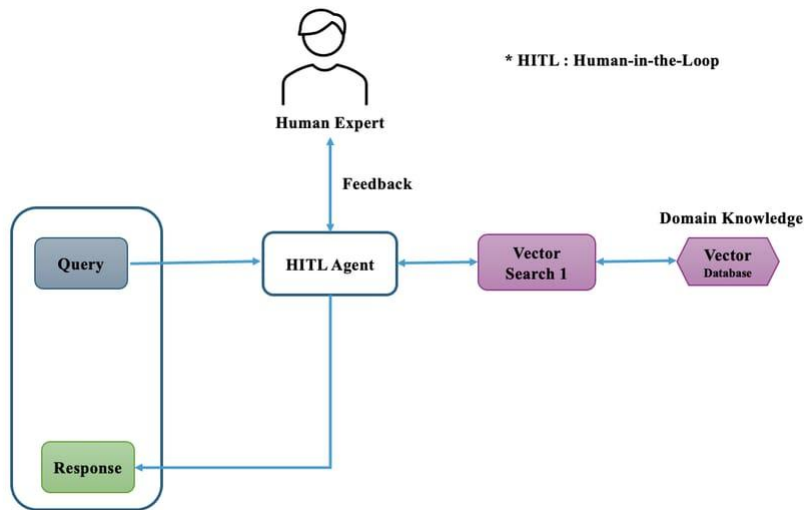
Human – In – The – Loop (HITL)



Human-in-the-Loop(HITL)

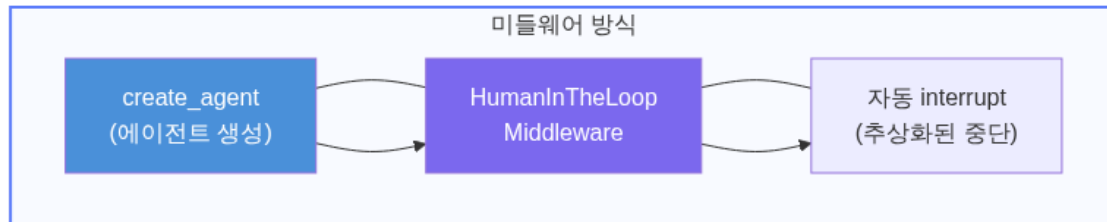
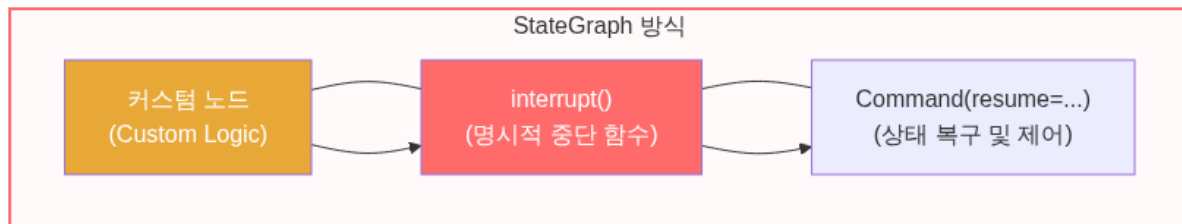
Human-in-the-Loop(HITL)은 에이전트의 실행을 **중간에 멈추고**, 사람의 승인을 받은 후 **재개**하는 패턴

- **고위험 작업**: 이메일 전송, 파일 삭제, 데이터베이스 수정 등 되돌리기 어려운 작업
- **불확실한 판단**: 검색 결과가 모호하거나 여러 해석이 가능한 경우
- **비용이 큰 작업**: 유료 API 호출, 대규모 데이터 처리 등
- **규정 준수**: 금융, 의료 등 사람의 최종 승인이 법적으로 요구되는 도메인



LangGraph에서 HITL을 구현하는 두 가지 방법

방법	적합한 상황	복잡도
HumanInTheLoopMiddleware	create_agent와 함께 사용, 간단한 승인	낮음
interrupt() + StateGraph	복잡한 승인 로직, 조건부 분기	중간



HITL은 승인만이 아니라 수정·보완까지 포함

사람이 중간에 개입하는 목적에 따라 UX와 State 설계가 달라집니다.

패턴	사용 상황	재개 시 필요한 값
Approve / Reject	메일 발송, DB 수정, 외부 전송	approved: true/false
Edit and Continue	초안·요약·리포트 검수	edited_content
Request More Info	입력 부족, 근거 불충분	additional_input

HITL 설계 질문: 어디서 멈출 것인가? 무엇을 보여줄 것인가? 어떤 값으로 재개할 것인가?

interrupt()와 Command(resume=...) 흐름

interrupt는 실행을 멈추고 상태를 저장하며, resume 값으로 그래프를 다시 진행합니다.



```
decision = interrupt({"draft": state["draft"]})  
  
Command(resume={"approved": True,  
               "edited_draft": "..."})
```

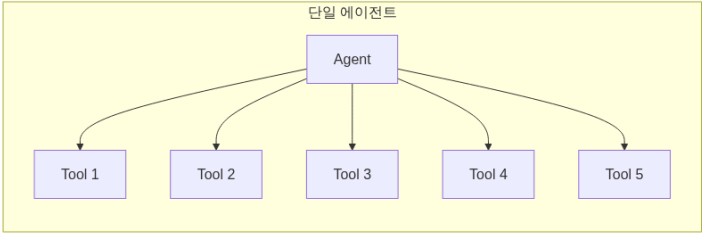
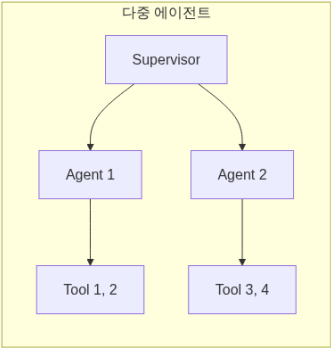
Multi Agent | 다중에이전트



다중 에이전트 시스템

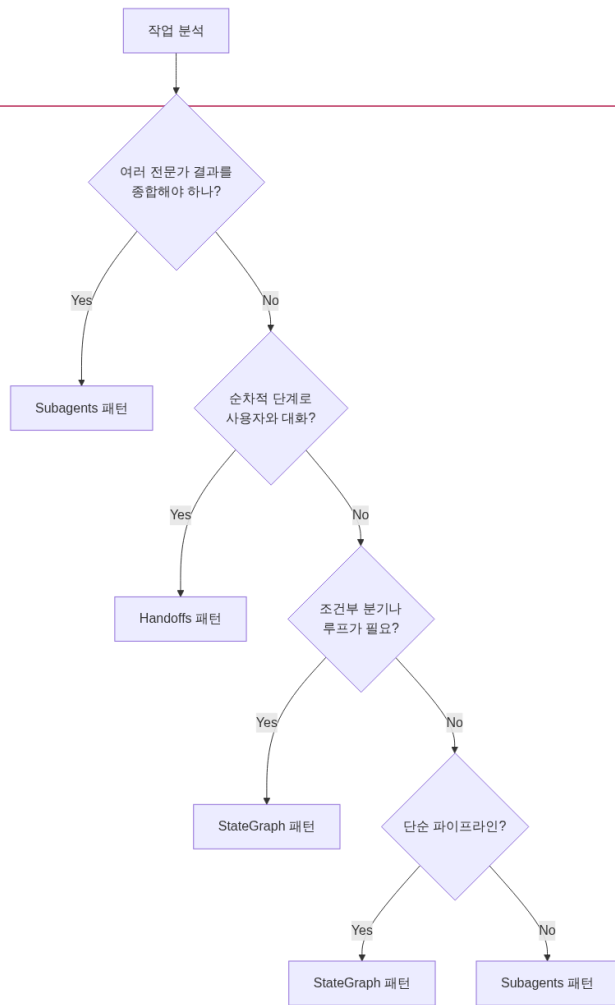
다중 에이전트 시스템은 하나의 에이전트로 모든 작업을 처리하는 대신, 전문화된 여러 에이전트가 협력하여 복잡한 문제를 해결하는 아키텍처

문제	단일 에이전트	다중 에이전트
도구 로딩	모든 도구를 한 번에 로드	각 에이전트가 필요한 도구만 로드
컨텍스트	불필요한 정보로 낭비	역할별로 효율적 사용
확장성	도구 추가 시 전체 영향	에이전트 단위 독립 확장
개발	한 명이 전체 관리	팀별 독립 개발 가능



다중 에이전트 아키텍처 패턴

기준	Subagents	Handoffs	StateGraph
제어 방식	Supervisor가 중앙 제어	상태 기반 분산 제어	그래프 엣지로 흐름 제어
사용자 대화	Supervisor만 대화	각 단계 에이전트가 직접 대화	노드별 처리
병렬 실행	Supervisor가 병렬 호출 가능	순차적 처리	Send로 병렬 가능
상태 관리	에이전트 내부 관리	AgentState + Command	TypedDict
적합한 경우	여러 전문가 결과 종합	순차적 워크플로우	복잡한 조건부 분기
구현 복잡도	낮음	중간	높음



Retrieve and Search



Retrieve & Search

항목

Retrieve

Search

데이터 소스

고정된 내부 데이터 (Vector DB)

실시간 외부 데이터 (Web)

주요 용도

전문 지식, 사내 보안 데이터 활용

최신 정보 확인, 일반 상식 보충

비용/속도

상대적으로 빠르고 저렴함

API 호출 비용 및 네트워크 지연 발생

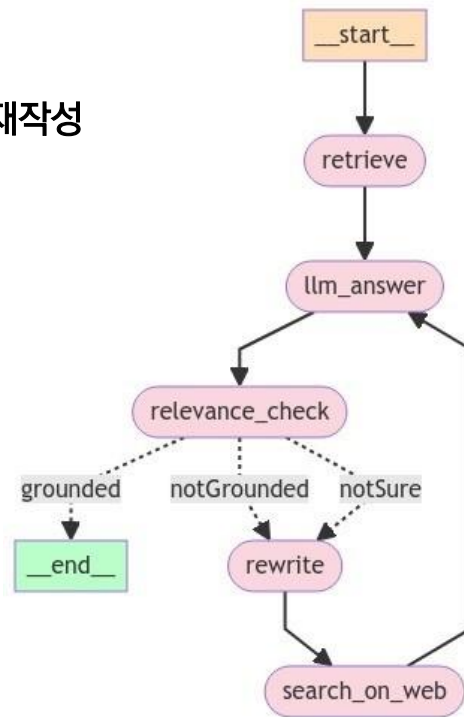
신뢰도

관리된 데이터이므로 신뢰도 높음

출처에 따라 신뢰도가 다를 수 있음

Retrieve and Search

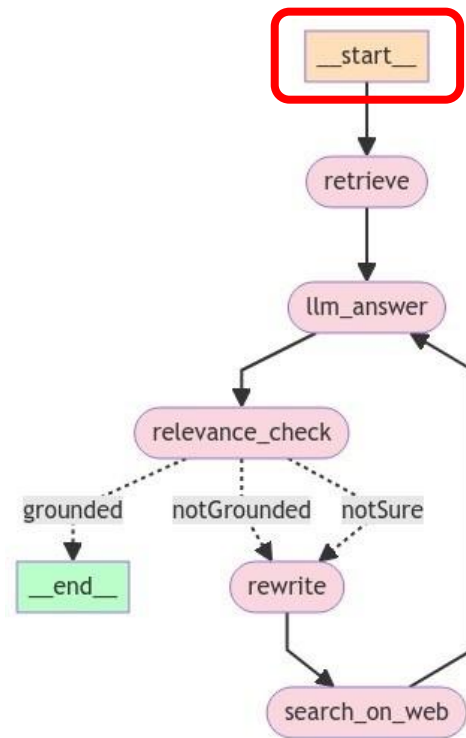
- 전통적인 RAG 를 수행
- 검색된 문서에 답변에 필요한 정보가 부족한 경우 “웹 검색”을 위한 쿼리 재작성
- “웹 검색” 으로 보충된 정보로 답변 도출 시도
- 새로운 답변으로 관련 체크 후 재조정/종료 진행



Retrieve and Search

- `_start_`: 질문 입력

“생성형 AI 가우스를 만든 회사의 2023년도 매출액은 얼마인가요?”



Retrieve and Search

- retrieve: PDF 문서에 대한 검색 수행
- “생성형 AI 가우스를 만든 회사의 2023년도 매출액은 얼마인가요?”

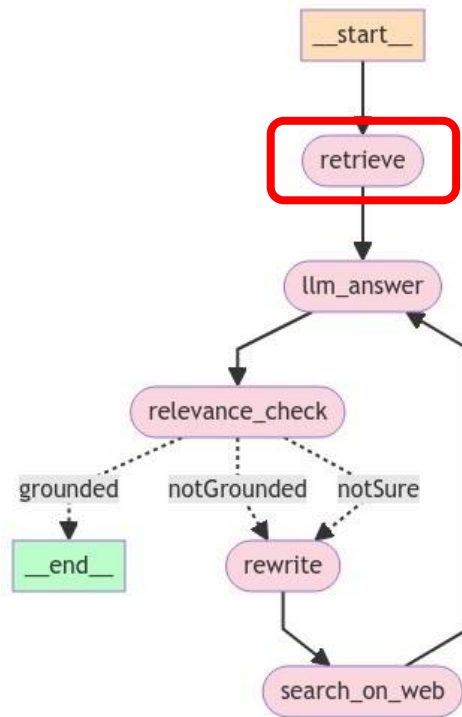
삼성전자, 자체 개발 생성 AI ‘삼성 가우스’ 공개

KEY Contents

- 삼성전자가 온디바이스에서 작동 가능하며 언어, 코드, 이미지의 3개 모델로 구성된 자체 개발 생성 AI 모델 ‘삼성 가우스’를 공개
- 삼성전자는 삼성 가우스를 다양한 제품에 단계적으로 탑재할 계획으로, 온디바이스 작동이 가능한 삼성 가우스는 외부로 사용자 정보가 유출될 위험이 없다는 장점을 보유

언어, 코드, 이미지의 3개 모델로 구성된 삼성 가우스, 온디바이스 작동 지원

- 삼성전자가 2023년 11월 8일 열린 ‘삼성 AI 포럼 2023’ 행사에서 자체 개발한 생성 AI 모델 ‘삼성 가우스’를 최초 공개
- 정규분포 이론을 정립한 천재 수학자 가우스(Gauss)의 이름을 본뜬 삼성 가우스는 다양한 상황에 최적화된 크기의 모델 선택이 가능
- 삼성 가우스는 라이선스나 개인정보를 침해하지 않는 안전한 데이터를 통해 학습되었으며, 온디바이스에서 작동하도록 설계되어 외부로 사용자의 정보가 유출되지 않는 장점을 보유
- 삼성전자는 삼성 가우스를 활용한 온디바이스 AI 기술도 소개했으며, 생성 AI 모델을 다양한 제품에 단계적으로 탑재할 계획



Retrieve and Search

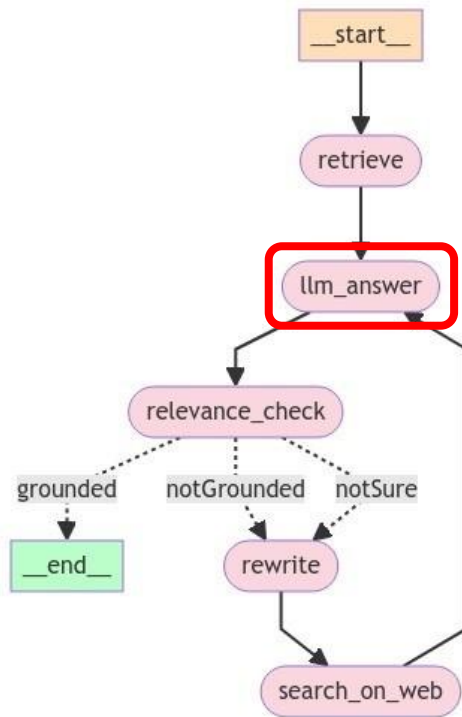
- llm_answer: 문서 기반 답변 도출 (GPT)
- “생성형 AI 가우스를 만든 회사의 2023년도 매출액은 얼마인가요?”

[답변]

생성형 AI '가우스'를 만든 회사는 삼성전자입니다.

그러나 제공된 문맥에서는 **삼성전자의 2023년도 매출액에 대한 정보는 언급되어 있지 않습니다.**

따라서 삼성전자의 2023년도 매출액에 대한 정보를 제공할 수 없습니다.



Retrieve and Search

- relevance_check: 관련/유효 체크
- “생성형 AI 가우스를 만든 회사의 2023년도 매출액은 얼마인가요?”

[답변]

생성형 AI '가우스'를 만든 회사는 삼성전자입니다.

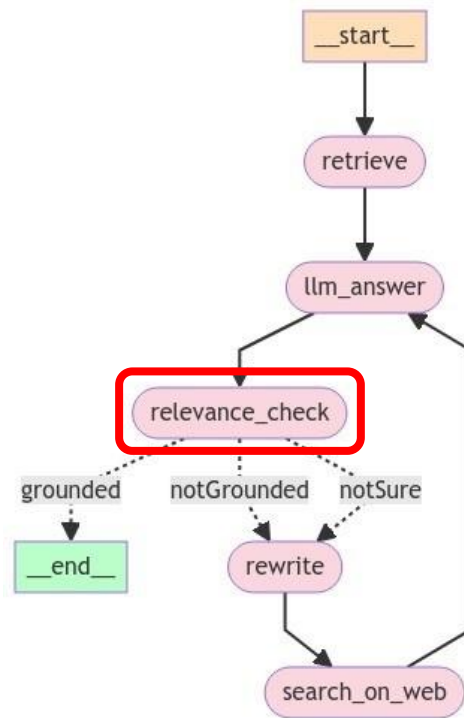
그러나 제공된 문맥에서는 **삼성전자의 2023년도 매출액에 대한**

정보는 언급되어 있지 않습니다.

따라서 삼성전자의 2023년도 매출액에 대한 정보를 제공할 수 없습니다.

이전 답변의 관련성/유효성 체크

“notGrounded”



Retrieve and Search

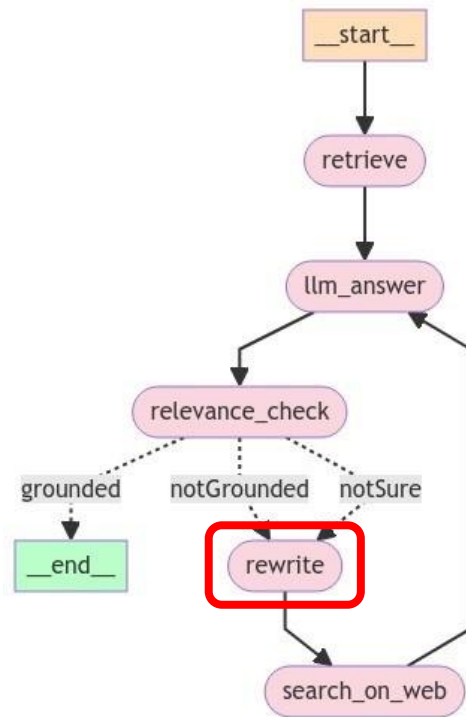
- rewrite: 추가정보 검색을 위한 쿼리 재작
- “생성형 AI 가우스를 만든 회사의 2023년도 매출액은 얼마인가요?”

rewrite: 쿼리 재작성

“Rewrite the question to get **additional information to get the answer**”

[결과]

“삼성전자의 2023년도 매출액은 얼마인가요?”



Retrieve and Search

- search_on_web: 재작성된 쿼리로 검색 수행
- “삼성전자의 2023년도 매출액은 얼마인가요?”

검색
결과

'웹 알림 동의 (크롬브라우저만 가능)\n'

'속보\n'

'삼성전자 2023년 매출 258조·영업이익 6.5조\n'

'4분기 잠정 실적...매출 67조·영업익 2.8조\n'

'서울 서초구 삼성전자 서초사옥에서 삼성 깃발이 바람에 휘날리고 있다. 이 기사와 관련된 기사\n'

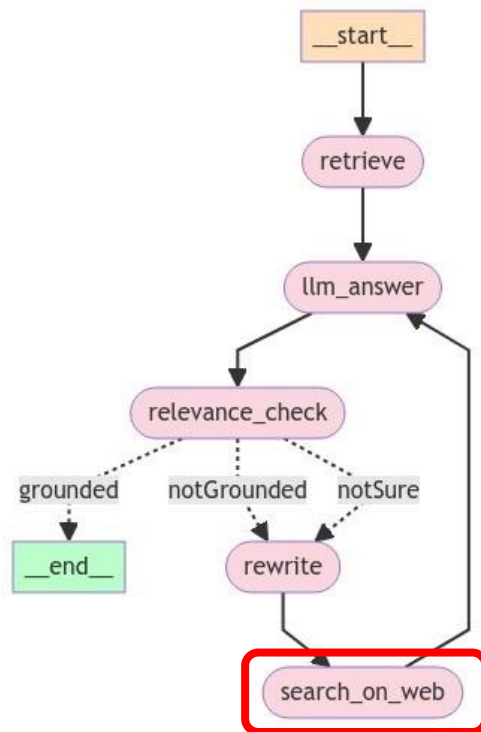
'당신이 관심 있을만한 이슈\n'

'경제 많이 본 뉴스\n'

'LIVE ISSUE\n'

'댓글0\n'

'댓글 신고\n'



Retrieve and Search

- search_on_web: 재작성된 쿼리로 검색 수행
- “삼성전자의 2023년도 매출액은 얼마인가요?”

[검색

'웹 알림 동의 (크롬브라우저만 가능)\n'

'속보\n'

'삼성전자 2023년 매출 258조·영업이익 6.5조\n'

'4분기 잠정 실적...매출 67조·영업익 2.8조\n'

'서울 서초구 삼성전자 서초사옥에서 삼성 깃발이 바람에 휘날리고 있다. 이 기사와 관련된 기사\n'

'당신이 관심 있을만한 이슈\n'

'경제 많이 본 뉴스\n'

'LIVE ISSUE\n'

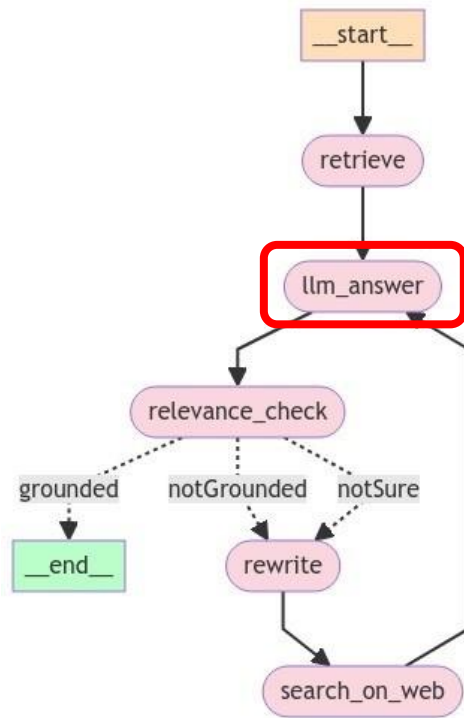
'댓글0\n'

'댓글 신고\n'

[답변]

삼성전자의 2023년도 매출액은 258.94조원입니다.

(출처: <https://news.samsung.com/kr/삼성전자-2023년-4분기-실적-발표>)



Retrieve and Search

- **relevance_check**: 관련성/유효성 체크
- “삼성전자의 2023년도 매출액은 얼마인가요?”

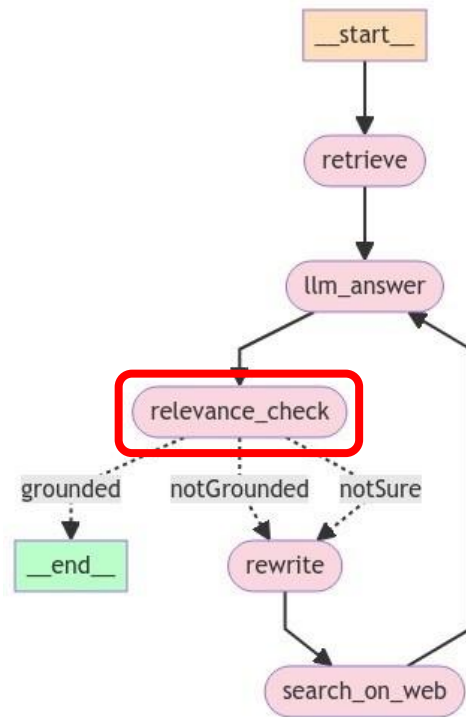
[답변]

삼성전자의 2023년도 매출액은 258.94조원입니다.

(출처: <https://news.samsung.com/kr/삼성전자-2023년-4분기-실적-발표>)

이전 답변의 관련성/유효성 체크

“grounded”



Retrieve and Search

- `_end_`: 종료

[질문]

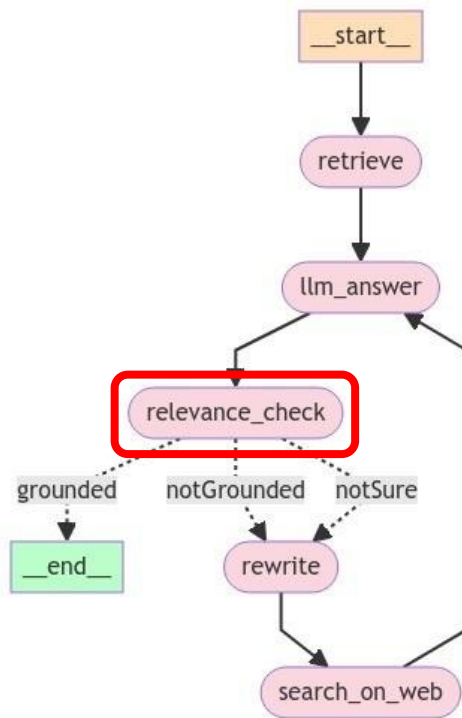
“생성형 AI 가우스를 만든 회사의 2023년도 매출액은 얼마인가요?”

[답변1]

삼성전자의 2023년도 매출액은 258.94조원입니다.

[답변2]

생성형 AI “가우스” 를 만든 회사는 삼성전자이며
2023년도 매출액은 258.94조원입니다



LangGraph – State(상태) / RAG 사례

❖ 노드 별 상태 값의 변화

① Question

Key	Value
context	
question	질문1
answer	
score	

노드1

② Retrieve

Key	Value
context	문서1
question	질문1
answer	
score	

노드2

③ Answer

Key	Value
context	문서1
question	질문1
answer	답변1
score	

노드3

④ Evaluate

Key	Value
context	문서1
question	질문1
answer	답변1
score	BAD

노드4

- 노드4에서 “문서”에 대하여 답변 관련성 점수를 부여
- score가 “BAD”인 경우 (선택할 수 있는 행동 3가지)
 - 노드 1: 질문을 재작성 요청
 - 노드 2: 문서를 다시 검색 / 검색을 통한 정보 보완
 - 노드 3: 답변을 재작성 요청

LangGraph – State(상태) / RAG 사례

❖ 노드 1: 질문을 재작성 요청

5 Question

Key	Value
context	문서1
question	질문2
answer	답변1
score	BAD1

노드1

6 Retrieve

Key	Value
context	문서2
question	질문1
answer	답변1
score	BAD1

노드2

7 Answer

Key	Value
context	문서2
question	질문2
answer	답변2
score	BAD1

노드3

8 Evaluate

Key	Value
context	문서2
question	질문2
answer	답변2
score	GOOD

노드4



Query Transform: 질문 재작성

LangGraph – State(상태) / RAG 사례

❖ 노드 2: 질문을 재작성 요청

1 Question

Key	Value
context	
question	질문1
answer	
score	

노드1

5 Retrieve

Key	Value
context	문서2
question	질문1
answer	답변1
score	BAD

노드2

6 Answer

Key	Value
context	문서2
question	질문1
answer	답변2
score	BAD

노드3

6 Evaluate

Key	Value
context	문서2
question	질문2
answer	답변2
score	GOOD

노드4



Context Retrieval 조정
(Chunk, 다른 검색기, web search, ...)

LangGraph – State(상태) / RAG 사례

❖ 노드 3: 답변을 재생성 요청

1 Question

Key	Value
context	
question	질문
answer	
score	

노드1

2 Retrieve

Key	Value
context	문서1
question	질문1
answer	
score	BAD

노드2

5 Answer

Key	Value
context	문서1
question	질문1
answer	답변2
score	BAD

노드3

6 Evaluate

Key	Value
context	문서1
question	질문2
answer	답변2
score	GOOD

노드4



프롬프트 조정 & 다른 LLM 사용
GPT, Claude, Local

Retrieve

Retrieve는 주로 **벡터 데이터베이스(Vector DB)**나 로컬 문서 저장소에서 사용자의 질문과 관련된 정보를 가져오는 과정입니다.

대표적인 워크플로우 패턴

LangGraph의 강점은 이 두 가지를 **조건부 에지(Conditional Edge)**로 엮어 똑똑한 흐름을 만드는 데 있습니다.

우선 순위 설정: * 먼저 Retrieve를 수행합니다.

평가 노드(Grader)에서 "문서가 질문에 충분한가?"를 판단합니다.

충분하면 답변 생성으로 가고, 부족하면 Search 노드로 이동합니다.

도구 선택(Tool Calling):

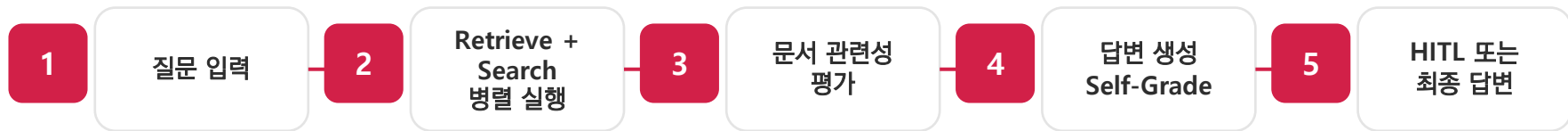
LLM 에이전트가 질문을 분석하여 retrieve_tool을 쓸지 web_search_tool을 쓸지 스스로 결정하게 합니다.

병렬 실행:

내부 데이터와 외부 데이터를 동시에 가져와서(Parallel Node) 종합적인 답변을 내놓습니다.

종합 실습 예시: 신뢰형 RAG Agent

앞의 개념을 하나의 실습으로 묶으면 수강생이 실제 설계 흐름을 이해하기 쉽습니다.



- State Reducer: documents, logs, messages 누적
- Command/Conditional Edge: grounded 여부에 따라 분기
- RetryPolicy: Web Search 실패 시 재시도 후 실패 노드 이동
- Trace/Test: 실행 경로와 State 변경을 확인
- Guardrail: 출처 없는 답변과 고위험 Tool 호출 제한

Q&A